Tangent-Actor: Learning to Linearize High-Dimensional Semilinear PDEs

Abstract

We introduce *Tangent-Actor*, a neural solver for high-dimensional semilinear PDEs that learns to linearize nonlinear equations before solving them. Our approach is motivated by a well-documented failure mode of full-history multilevel Picard (FH-MLP) methods: repeatedly applying nonlinear drivers to noisy Monte Carlo estimates, particularly of high-dimensional gradients, can amplify errors. Instead of relying on this nonlinear recursion, Tangent-Actor learns state-dependent linearizations that turn each evaluation into a linear PDE. The resulting equations can be solved through Feynman–Kac expectations, providing value and gradient estimates that guide successive actor updates. The learned gradient actor also serves as a martingale control variate, reducing Monte Carlo variance without an additional network. We establish conditional error bounds through actor consistency, characterize residual Monte Carlo variance, and prove ideal improvement for convex or concave drivers. Experiments across diverse high-dimensional PDEs, including nonconvex coupled problems, demonstrate strong accuracy against established baselines.

1 Introduction

High-dimensional semilinear parabolic partial differential equations (PDEs) arise in stochastic control, financial mathematics, and systems evolving under uncertainty and are difficult to solve (Fleming and Soner, 2006; Pham, 2009). Figure 5 illustrates this complexity: even two-dimensional slices of high-dimensional PDE solutions can exhibit rich spatial structures. Traditional numerical methods, including finite difference, finite volume, and finite element methods (LeVeque, 2007; Eymard et al., 2000; Brenner and Scott, 2008), often suffer from the curse of dimensionality. Multilevel Picard (MLP) methods offer a remarkable alternative: by combining Picard iteration with Monte Carlo sampling, they solve high-dimensional PDEs without constructing spatial grids and provide rigorous polynomial-complexity guarantees for important classes of semilinear equations (Hutzenthaler et al., 2020; Hutzenthaler and Kruse, 2020).

Despite these advances, full-history MLP can still suffer from the curse of dimensionality. Hutzenthaler and Nguyen (2025) established such a failure even for a PDE whose exact solution depends on only one spatial coordinate. This example reveals a striking difficulty: increasing the ambient dimension does not make the true solution more complicated, yet it can make its stochastic approximation substantially less accurate.

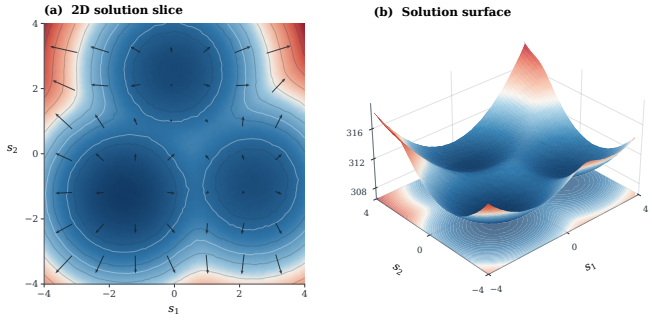


Figure 1: An Example Solution by Tangent-Actor.

The intuition lies in how nonlinearity interacts with Monte Carlo noise. For example, a recursive Feynman–Kac method may need to estimate a squared gradient norm $\|z\|^2$. Even if the Monte Carlo estimate $\hat{z} = z + \varepsilon$ is unbiased, with $\mathbb{E}[\varepsilon] = 0$, we have

$$\mathbb{E}[\|\hat{z}\|^2] = \|z\|^2 + \mathbb{E}[\|\varepsilon\|^2].$$

Thus, applying a nonlinear function to an unbiased estimate can introduce systematic bias. This bias can grow with dimension as noise accumulates across gradient coordinates, and averaging more evaluations does not eliminate it at a fixed inner sampling budget. Repeated nonlinear evaluations can then propagate these errors through the Picard recursion. Moreover, stochastic evaluation introduces variance even in the absence of such bias. Both effects can reduce accuracy when errors propagate through successive Picard iterations. Figure 2 illustrates this dimensional instability: the RMSE of the evaluated full-history MLP estimator is $909\times$ that of Tangent-Actor at $d = 50$, increasing to $6{,}803\times$ at $d = 400$, even though the exact solution depends on only one coordinate.

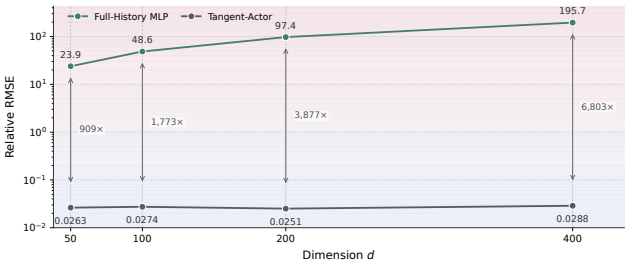


Figure 2: Dimension scaling on the MLP counterexample.

In contrast, a fixed affine function preserves unbiasedness under expectation. This suggests a simple principle: linearize the nonlinear driver before applying Monte

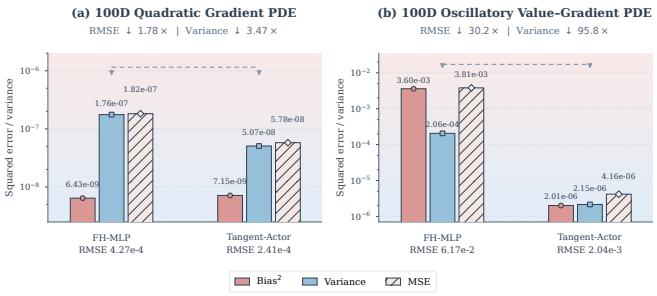


Figure 3: Bias–variance decomposition on two PDEs.

Carlo, rather than applying it to noisy estimates. We introduce *Tangent-Actor*, a neural PDE solver that learns state-dependent linearizations and evaluates the resulting linear PDEs through Feynman–Kac expectations. This avoids nested nonlinear evaluations of noisy Monte Carlo estimates, replacing them with a nonrecursive stochastic expectation.

The learned gradient actor is also reused as a martingale control variate to reduce sampling variance, without requiring an additional network. Figure 3 illustrates how these two components work together. When the two methods have comparable bias, as in the quadratic-gradient PDE, Tangent-Actor produces a substantially lower-variance estimator. When FH-MLP exhibits much larger bias, as in the oscillatory value–gradient PDE, Tangent-Actor achieves both lower bias and lower variance. Thus, the benefit is not limited to correcting nonlinear bias: reducing stochastic fluctuations also improves accuracy when bias is already small. The comparison uses 64 fixed query states and 100 independent stochastic replicates under fixed configurations.

The Feynman–Kac representation is central to our algorithm and analysis. Appendix A provides a one-dimensional introduction for readers unfamiliar with this representation.

1.1 Our Contributions

Our contributions are threefold:

(1) **Neural PDE solver.** We explain how nonlinear evaluation of noisy Monte Carlo gradient estimates can introduce dimension-dependent bias, shedding light on a documented failure mode of full-history MLP. Motivated by this observation, we introduce *Tangent-Actor*. Our method unifies learned linearization and martingale variance reduction in a terminal-aware neural solver, without requiring ground-truth PDE solution labels.

(2) **Theoretical analysis.** We derive exact identities for the linearization error and residual Monte Carlo variance, establish conditional second-order error bounds in the tangent defect, and characterize ideal actor improvement for convex or concave drivers.

(3) **Empirical evaluation.** We demonstrate strong accuracy across high-dimensional semilinear PDEs, including HJB, Allen–Cahn, and coupled nonlinear equations. Comparisons with Deep Picard and PINN-based baselines, together with a known FH-MLP failure example, validate the effectiveness of learned linearization across diverse nonlinearities and dimensions.

2 Related Work

Multilevel Picard and stochastic fixed-point methods. Multilevel Picard (MLP) methods combine stochastic representations, Picard iteration, and multilevel Monte Carlo to solve high-dimensional semilinear PDEs without spatial grids, with rigorous complexity guarantees under suitable assumptions (E et al., 2021; Hutzenthaler and Kruse, 2020). However, nonlinear recursive evaluations can propagate sampling errors, and a recent counterexample establishes that full-history MLP can still suffer from the curse of dimensionality (Hutzenthaler and Nguyen, 2025). Alternative approaches to unbiased estimation include branching diffusions for polynomial nonlinearities (Henry-Labordère et al., 2019) and randomized debiasing techniques (Rhee and Glynn, 2015; Blanchet and Glynn, 2015). Deep Picard Iteration (DPI) (Han et al., 2026) instead uses neural regression on solution values and gradients, with control variates for gradient estimation. Tangent-Actor learns linearizations of the nonlinear driver, allowing each iteration to be evaluated through a linear Feynman–Kac representation.

Neural solvers for high-dimensional PDEs. Regression-based Monte Carlo methods have long been used to solve backward stochastic differential equations (BSDEs) (Gobet et al., 2005). Deep BSDE trains neural networks to match terminal conditions along simulated trajectories (Han et al., 2018), whereas deep backward dynamic programming (Huré et al., 2020) and deep splitting (Beck et al., 2021) solve backward sequences of local regressions with one network per time step; regression errors propagate through this induction (Germain et al., 2022), and refining the time grid requires retraining the whole chain. Physics-informed neural networks (PINNs) approximate solutions by minimizing PDE residuals (Raissi et al., 2019). For high-dimensional PDEs, Hutchinson trace estimation (HTE) reduces the cost of second-order derivatives (Hu et al., 2024a), while stochastic dimension gradient descent (SDGD) samples derivative coordinates (Hu et al., 2024b). Tangent-Actor instead linearizes the driver around a learned actor over the whole horizon, so its nonlinear residual is second order in the actor mismatch rather than tied to a step size, and stochastic solutions of the linearized PDEs supply supervised targets.

Policy iteration and neural linearization. Successive linearization has a long history in quasilinearization and Newton-type methods (Bellman and Kalaba, 1965), as well as policy iteration (Bokanowski et al., 2009; Ito et al., 2021). Neural policy iteration has also been developed for optimal control, including PINN-based evaluation of linear PDEs (Meng et al., 2024), value-gradient learning (Bensoussan et al., 2023), and actor–critic methods (Cohen et al., 2025).

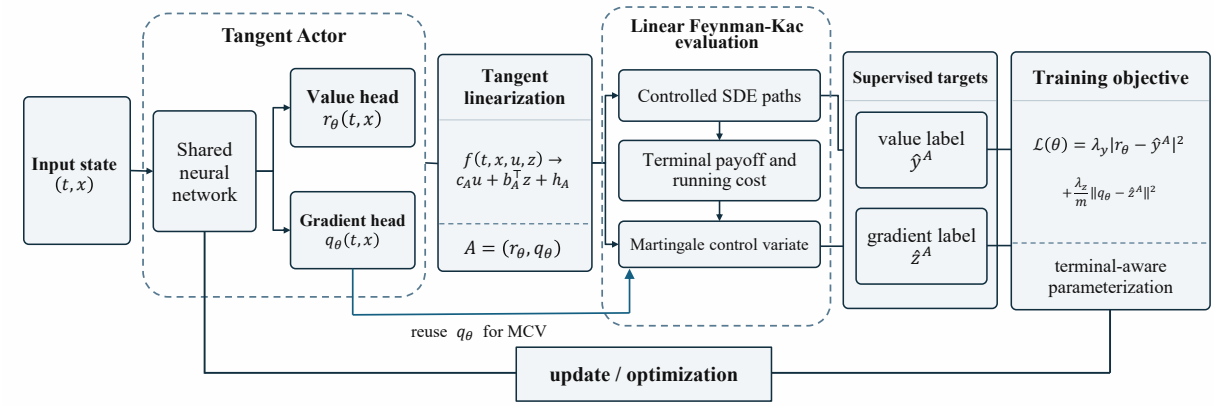


Figure 4: One actor-refinement iteration of Tangent-Actor.

Our approach learns tangent approximations of the general semilinear driver $f(u, z)$, without requiring a physical control interpretation. Each linearized PDE is evaluated through weighted Feynman–Kac expectations, whose values and gradients supervise the next actor. For convex or concave drivers, this construction also connects actor approximation error to PDE solution error.

Martingale control variates. Martingale-based variance reduction is well established in stochastic PDE solvers. Learned solution gradients have been used as control variates for linear Feynman–Kac estimators (Sabate Vidales et al., 2022), with related SDE-based approaches analyzed in Richter and Berner (2022). DPI also employs control variates for gradient estimation (Han et al., 2026). Tangent-Actor reuses the learned actor itself as the martingale integrand for each linearized PDE. Differentiating the control-variate-corrected payoff provides reduced-variance supervision for both values and gradients without a separate network, while actor approximation and time-discretization errors remain distinct.

3 Tangent-Actor Method

Tangent-Actor alternates between learning a tangent-linear PDE and using its stochastic solution to refine the tangent actor. After training, the selected actor is reused to evaluate the PDE at new query states, without further optimization. Figure 4 illustrates one actor-refinement iteration, while Algorithms 1 and 2 summarize training and inference.

3.1 Learning a Tangent-Linear PDE

We consider a high-dimensional semilinear parabolic PDE

$$\begin{aligned} \partial_t u + \mathcal{L}u + f(t,x,u,z_u) = 0, \\ z_u = \sigma^\top \nabla u, \qquad u(T,x) = g(x), \end{aligned} \tag{1}$$

where $x \in \mathbb{R}^d$ and $\mathcal{L}u = \mu^\top \nabla u + \frac{1}{2}\operatorname{Tr}(\sigma\sigma^\top \nabla^2 u)$. The difficulty is that the driver f depends on the unknown value–gradient pair (u, z_u). Rather than repeatedly evaluate this nonlinearity on noisy estimates, we learn *where* to linearize it.

At each state (t, x), one neural network outputs $A_\theta(t,x) = (r_\theta, q_\theta)$, specifying tangent points for the value and gradient arguments. We build known terminal data into these heads: for smooth g, $r_\theta = g + (T-t)R_\theta$ and $q_\theta = z_g + (T-t)Q_\theta$, with $z_g(x) = \sigma(T,x)^\top \nabla g(x)$. A bounded version is used when a valid value interval is available; a head may be omitted if the driver does not need it.

Let $Y = (y, z)$, and write $f(t, x, Y)$ for the driver. For a frozen actor $A = (r, q)$, its tangent is

$$\begin{aligned} \mathcal{T}_A(Y) &= f(t,x,A) + \nabla_Y f(t,x,A)^\top (Y - A) \\ &= c_A y + b_A^\top z + h_A, \end{aligned} \tag{2}$$

where $c_A = f_y(t,x,A)$, $b_A = \nabla_z f(t,x,A)$, and $h_A = f(t,x,A) - c_A r - b_A^\top q$. Freezing A makes these coefficients known functions of (t, x), so that we solve the *linear* PDE

$$\begin{aligned} \partial_t v^A + \mathcal{L}v^A + c_A v^A + b_A^\top z_{v^A} + h_A = 0, \\ v^A(T,x) = g(x). \end{aligned} \tag{3}$$

Why learn the tangent rather than fix it once? For a differentiable concave scalar function, every tangent is an upper bound, and

$$\phi(y) = \min_a \{\phi(a) + \phi'(a)(y-a)\}.$$

Similarly, for jointly concave or convex drivers, the choice of tangent controls the gap between the linearized and original equations. For general smooth drivers the tangent remains a local approximation, even without a global bound. In particular, an actor consistent with its own linearized PDE recovers the original equation; Section 4 makes this precise and bounds the error of approximate consistency. Choosing the tangent from a previous value–gradient approximation is also closely related to a Newton update.

3.2 Stochastic Evaluation and Supervision

The benefit of freezing the actor is that (3) admits a non-recursive Feynman–Kac representation. Define the actor-dependent drift $\mu_A = \mu + \sigma b_A$ and simulate

$$\begin{aligned} dX_s^A &= \mu_A(s, X_s^A)\, ds + \sigma(s, X_s^A)\, dW_s, \\ X_t^A &= x. \end{aligned} \tag{4}$$

With $C_s^A = \exp(\int_t^s c_A(\ell, X_\ell^A)\, d\ell)$, Itô's formula and (3) give

$$\begin{aligned} d\big(C_s^A v^A(s, X_s^A)\big) = &- C_s^A h_A(s, X_s^A)\, ds \\ &+ C_s^A z_{v^A}(s, X_s^A)^\top dW_s. \end{aligned}$$

The stochastic integral has zero expectation, yielding

$$\begin{aligned} v^A(t,x) = \mathbb{E}_{t,x}\Big[& C_T^A g(X_T^A) \\ & + \int_t^T C_s^A h_A(s, X_s^A)\, ds \Big]. \end{aligned} \tag{5}$$

Unlike a nonlinear Picard recursion, this expectation contains only known coefficients and the terminal data.

It still has Monte Carlo noise. The Itô identity shows that its fluctuations involve the unknown gradient $z_{v^A} = \sigma^\top \nabla v^A$. The same actor gradient q used to define the tangent is a candidate approximation to this quantity. We reuse it as a predictable martingale control variate, without training another network:

$$\begin{aligned} \widetilde{P}^A = C_T^A g(X_T^A) + \int_t^T C_s^A h_A(s, X_s^A)\, ds \\ - \int_t^T C_s^A q(s, X_s^A)^\top\, dW_s. \end{aligned} \tag{6}$$

The correction has zero mean, so $\mathbb{E}_{t,x}[\widetilde{P}^A] = v^A(t,x)$ under standard integrability conditions. Its variance is small when q approximates z_{v^A} along the simulated paths; this reduction is not guaranteed for an arbitrary actor (Section 4).

Crucially, the *same corrected trajectories* provide both kinds of supervision. For M independent paths,

$$\widehat{y}^A = \frac{1}{M}\sum_{i=1}^M \widetilde{P}_i^A, \qquad \widehat{z}^A = \sigma(t,x)^\top \nabla_x \widehat{y}^A. \tag{7}$$

We obtain the gradient label by pathwise differentiation: Brownian increments and actor parameters are held fixed, while the state dependence of the diffusion, weights, source terms, and correction remains differentiable.

3.3 Actor Refinement and Inference

Given labels from the frozen actor A_j, we train $A_\theta = (r_\theta, q_\theta)$ to obtain the next actor A_{j+1} by minimizing

$$\begin{aligned} \mathcal{J}_j(\theta) = \mathbb{E}_{(t,x)\sim\nu}\Big[& \lambda_y |r_\theta - \widehat{y}^{A_j}|^2 \\ & + \frac{\lambda_z}{m}\|q_\theta - \widehat{z}^{A_j}\|^2 \Big], \end{aligned} \tag{8}$$

Algorithm 1: Tangent-Actor training

Require: PDE $(\mathcal{L}, f, g)$, distribution ν, rounds J
1: Initialize terminal-aware actor A_0
2: **for** $j = 0, \dots, J-1$ **do**
3: Freeze A_j and form its tangent-linear PDE
4: Sample $(t,x) \sim \nu$; form labels (7)
5: Fit A_{j+1} using (8)
6: **end for**
7: Select $A_\star$ using reference-free validation
8: **return** $A_\star$

Algorithm 2: Tangent-Actor inference

Require: Actor $A_\star$, query (t,x), paths M_{eval}
1: Freeze $A_\star$; simulate fresh FK–MCV payoffs
2: $\widehat{u}(t,x) \leftarrow M_{\text{eval}}^{-1} \sum_{i=1}^{M_{\text{eval}}} \widetilde{P}_i^{A_\star}$
3: **return** $\widehat{u}(t,x)$

where ν is the training-state distribution and m is the dimension of z_u. The newly fitted actor then defines the next linear PDE. Thus, the method iteratively learns from PDE solves generated by its own previous tangent, without ground-truth solution labels.

After training, the actor is reused for new states rather than retrained for every query. At a new (t,x), we freeze the selected actor $A_\star$ and use fresh trajectories to estimate the solution of its linearized PDE. The final prediction is the FK–MCV average, rather than simply the actor's value head; no further actor updates are needed.

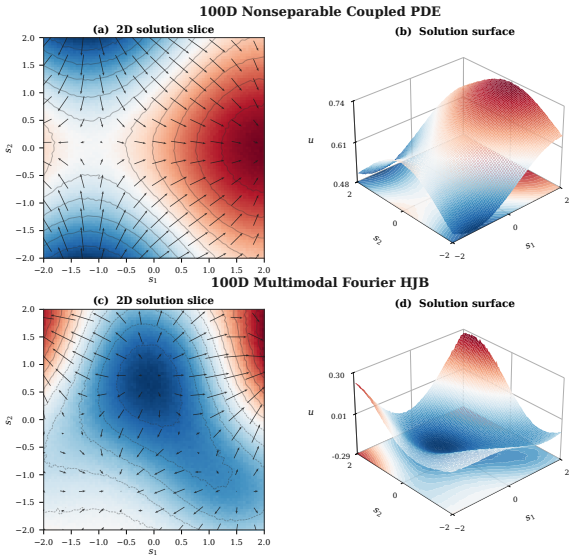


Figure 5: An Example Solution by Tangent-Actor.

4 Why Does Tangent-Actor Work?

Tangent-Actor may appear self-referential: the actor defines a linear PDE and is then trained on its solution. Why should this recover the original nonlinear equation? Tangency provides the link: exact self-consistency recovers the nonlinear PDE, while an imperfect actor leaves a second-order residual. The same mismatch also controls the remaining Monte Carlo variance. Formal assumptions and proofs are in Appendix B.

4.1 From Actor Consistency to PDE Accuracy

Fix an actor $A = (r, q)$, where r and q are its value and gradient outputs. Let v^A solve the linear PDE defined by A in (3). Define the value–gradient pair returned by that linear PDE and its discrepancy from the actor:

$$Y^A = (v^A, z_{v^A}), \qquad \delta_A = Y^A - A,$$

where $z_{v^A} = \sigma^\top \nabla v^A$. The *actor-consistency defect* $\delta_A = (v^A - r, z_{v^A} - q)$ measures whether the actor agrees with its own linear PDE. We omit the shared (t, x) arguments in f and $\mathcal{T}_A$.

If $\delta_A = 0$ at every state, tangency gives

$$\mathcal{T}_A(Y^A) = \mathcal{T}_A(A) = f(A) = f(Y^A).$$

Thus v^A also solves the original PDE; uniqueness implies $v^A = u$.

In practice, δ_A is not zero. The *nonlinear PDE residual* of v^A satisfies

$$\mathcal{R}_A = f(Y^A) - \mathcal{T}_A(Y^A), \tag{9}$$

$$|\mathcal{R}_A| \le \frac{H}{2}\|\delta_A\|^2. \tag{10}$$

The second line follows from Taylor's theorem: the linear error vanishes by tangency. Here H bounds the Hessian norm of f with respect to (y, z) between A and Y^A.

Under suitable PDE stability assumptions, this residual controls the solution error:

$$\|v^A - u\|_\infty \le C_T\|\mathcal{R}_A\|_\infty \le \frac{C_T H}{2}\|\delta_A\|_\infty^2. \tag{11}$$

Here $\|\cdot\|_\infty$ is the supremum over (t, x) (with Euclidean norm for vectors), and C_T is a stability constant; one may take $C_T = Te^{L_y T}$ if $|f_y| \le L_y$ under the stated assumptions. Thus a small defect implies a small solution error without ground-truth labels. This argument requires smoothness and stability, not convexity; finite validation states cannot certify the global bound.

For jointly concave drivers, the ideal update $A_{j+1} = Y^{A_j}$, using exact linear PDE solutions, also satisfies

$$u(t, x) \le v^{A_{j+1}}(t, x) \le v^{A_j}(t, x). \tag{12}$$

Each ideal step tightens an upper bound on u. The inequalities reverse for jointly convex drivers; learned updates need not be monotone.

4.2 Bias and Variance of Stochastic Supervision

The preceding bounds concern approximation error from an imperfect tangent. Evaluating the resulting linear PDE adds Monte Carlo noise. Its Feynman–Kac representation (5) avoids applying the nonlinear driver to nested noisy estimates. Moreover, the martingale correction (6) leaves the mean unchanged: $\mathbb{E}^A_{t,x}[\widetilde{P}^A] = v^A(t, x)$. It targets sampling variance, not tangent or time-discretization bias.

Let $\delta_{A,z} = z_{v^A} - q$ denote the gradient component of the consistency defect. For the controlled path X^A_s in (4), with weight $C^A_s = \exp(\int_t^s c_A(\ell, X^A_\ell)\, d\ell)$, Itô isometry gives the continuous-time identity

$$\mathrm{Var}^A_{t,x}(\widetilde{P}^A) = \mathbb{E}^A_{t,x}\left[\int_t^T (C^A_s)^2 \times \|\delta_{A,z}(s, X^A_s)\|^2\, ds\right]. \tag{13}$$

Here $\mathbb{E}^A_{t,x}$ and $\mathrm{Var}^A_{t,x}$ are over Brownian paths starting at $X^A_t = x$, conditional on the frozen actor. Averaging M independent payoffs divides this variance by M. Thus the gradient part of actor inconsistency also determines the remaining noise in value supervision. If q is poor, MCV can increase variance; this identity does not guarantee variance reduction for pathwise gradient labels.

5 Experiments

We evaluate Tangent-Actor in two complementary settings: predicting an entire solution function at unseen states, and solving individual PDE queries. The benchmarks span $d \in \{20, 50, 100, 200, 400\}$ and include gradient-only, value-only, convex/concave, and nonconvex value–gradient drivers. We compare predictions $\widehat{u}_i$ with analytic or independently refined numerical references $u^\star_i$. We report value RMSE and $\mathrm{RelMAE} = \sum_i |\widehat{u}_i - u^\star_i| / \sum_i |u^\star_i|$; lower is better. Each method is evaluated on the same frozen test states within a given PDE and dimension. Unless stated otherwise, the results are fixed-configuration accuracy comparisons rather than matched-compute comparisons.

5.1 Whole-function Accuracy

Baselines and implementation settings. We compare Tangent-Actor against Deep Picard Iteration (DPI), PINN-HTE, and SDGD-PINN using their published algorithms and source implementations.

For DPI, we follow the gradient-supervised semilinear setting of Han et al. (2026) (Section 5.2), using 80 Picard iterations, 4,096 training states and Monte Carlo samples, 16 training epochs per iteration, and a four-layer, width-128 ELU network. PINN-HTE follows the corresponding PINN baseline in the same implementation, with 30,000 training epochs, 16 Hutchinson probes, and a soft terminal penalty of weight 10. For the OU–Gaussian Mixture HJB, we instead retain the authors' specialized PISGradNet configurations, including 40 Picard iterations for DPI and 20,000 epochs for PINN-HTE. For SDGD-PINN, we follow

Table 1: Whole-function accuracy across eight PDEs (160 method cells). Within each method, the upper row is RMSE and the lower row is RelMAE; best values are bolded.

Method	Radial Log-Terminal HJB					OU–Gaussian Mixture HJB				
Dimension	20	50	100	200	400	20	50	100	200	400
Tangent-Actor	**9.48e-4**	**5.68e-4**	**8.14e-4**	**1.46e-3**	**9.24e-4**	**6.18e-2**	**1.36e-1**	**2.20e-1**	**1.40e0**	**5.16e1**
	2.40e-4	**1.07e-4**	**1.25e-4**	**1.63e-4**	**8.81e-5**	**8.28e-4**	**7.37e-4**	**5.98e-4**	**1.21e-3**	**2.67e-2**
DPI	1.49e-2	3.61e-2	1.34e-1	3.14e-1	3.50e-1	1.57e0	3.06e0	1.77e1	9.05e1	1.45e2
	3.61e-3	5.73e-3	1.74e-2	4.66e-2	4.88e-2	1.64e-2	1.41e-2	3.94e-2	9.81e-2	8.78e-2
PINN-HTE	1.31e-2	4.23e-2	5.36e-2	3.48e-1	4.75e-1	6.49e-1	4.32e0	3.87e1	9.66e1	2.23e2
	3.61e-3	7.51e-3	9.53e-3	5.36e-2	6.48e-2	7.21e-3	1.48e-2	8.74e-2	1.08e-1	1.31e-1
SDGD-PINN	6.09e-3	4.66e-2	3.99e-2	3.88e-2	2.92e-2	9.93e0	2.71e1	6.25e1	1.51e2	3.66e2
	1.48e-3	8.66e-3	6.07e-3	4.81e-3	3.20e-3	1.12e-1	1.43e-1	1.72e-1	2.08e-1	2.52e-1

Method	Concave Cubic–Quadratic PDE					Convex Cubic–Quadratic PDE				
Dimension	20	50	100	200	400	20	50	100	200	400
Tangent-Actor	**3.14e-4**	**3.25e-4**	**3.23e-4**	**3.30e-4**	**3.57e-4**	**2.14e-4**	**2.07e-4**	**2.12e-4**	**2.32e-4**	**2.63e-4**
	3.57e-4	**3.73e-4**	**3.71e-4**	**3.76e-4**	**3.94e-4**	**3.59e-4**	**3.37e-4**	**3.52e-4**	**3.88e-4**	**4.38e-4**
DPI	8.81e-4	9.67e-4	2.14e-3	2.76e-3	6.82e-3	7.28e-4	9.71e-4	1.86e-3	2.93e-3	4.09e-3
	1.16e-3	1.27e-3	3.26e-3	3.70e-3	9.29e-3	1.36e-3	1.76e-3	3.46e-3	5.58e-3	7.37e-3
PINN-HTE	3.83e-3	2.80e-3	3.23e-3	3.43e-3	5.26e-3	1.62e-3	2.68e-3	3.27e-3	4.60e-3	5.06e-3
	5.72e-3	3.99e-3	4.20e-3	4.73e-3	7.27e-3	2.89e-3	5.07e-3	6.11e-3	8.69e-3	8.93e-3
SDGD-PINN	2.59e-2	1.80e-2	2.33e-2	2.79e-2	2.97e-2	1.33e-2	1.50e-2	1.28e-2	1.68e-2	1.38e-2
	2.96e-2	2.11e-2	2.77e-2	3.70e-2	3.95e-2	2.29e-2	2.44e-2	2.18e-2	2.72e-2	2.21e-2

Method	Oscillatory Value–Gradient PDE					Allen–Cahn				
Dimension	20	50	100	200	400	20	50	100	200	400
Tangent-Actor	**8.13e-4**	**8.37e-4**	**8.04e-4**	**8.87e-4**	**1.26e-3**	**1.03e-3**	**4.29e-4**	**1.73e-4**	**6.41e-5**	**2.15e-5**
	1.34e-3	**1.33e-3**	**1.33e-3**	**1.42e-3**	**1.87e-3**	**3.96e-3**	**3.30e-3**	**2.57e-3**	**1.84e-3**	**1.24e-3**
DPI	1.08e-3	1.08e-3	1.70e-3	5.22e-3	6.15e-3	1.45e-3	1.44e-3	1.07e-3	9.04e-4	4.55e-4
	1.65e-3	1.64e-3	2.80e-3	8.05e-3	1.05e-2	6.03e-3	1.31e-2	1.83e-2	2.92e-2	3.02e-2
PINN-HTE	7.09e-3	4.27e-3	3.07e-3	3.67e-3	4.93e-3	1.72e-3	1.31e-3	9.48e-4	8.47e-4	4.26e-4
	1.43e-2	7.88e-3	5.47e-3	6.18e-3	8.35e-3	7.18e-3	1.09e-2	1.58e-2	2.63e-2	2.66e-2
SDGD-PINN	1.27e-2	1.37e-2	8.88e-3	1.04e-2	1.33e-2	1.19e-2	6.12e-3	2.69e-3	1.42e-3	6.16e-4
	1.24e-2	1.85e-2	1.17e-2	1.50e-2	2.05e-2	4.65e-2	4.99e-2	4.19e-2	4.41e-2	3.86e-2

Method	Full-Rank Log-Sum-Exp HJB					Fourier–Tanh Coupled PDE				
Dimension	20	50	100	200	400	20	50	100	200	400
Tangent-Actor	3.57e-4	**7.97e-4**	**7.04e-4**	**2.02e-3**	**1.78e-3**	**2.35e-3**	**2.22e-3**	**1.65e-3**	**4.27e-3**	**5.09e-3**
	4.25e-4	**5.74e-4**	**4.78e-4**	**7.92e-4**	**6.71e-4**	**2.82e-3**	**2.61e-3**	**8.08e-4**	**5.88e-3**	**6.81e-3**
DPI	8.50e-3	4.77e-2	1.43e-1	4.78e-1	8.47e-1	2.06e-2	6.25e-2	6.46e-2	2.46e-1	3.90e-1
	1.19e-2	4.42e-2	1.39e-1	2.87e-1	5.04e-1	2.73e-2	8.34e-2	3.75e-2	4.56e-1	7.10e-1
PINN-HTE	7.33e-3	3.36e-2	1.92e-1	7.53e-1	1.37e0	4.90e-2	6.55e-2	7.71e-2	2.62e-1	4.01e-1
	1.03e-2	2.89e-2	1.89e-1	4.56e-1	8.33e-1	6.48e-2	8.95e-2	4.33e-2	4.75e-1	7.21e-1
SDGD-PINN	**2.43e-4**	2.23e-3	1.13e-2	7.49e-2	9.41e-2	1.35e-2	5.13e-2	7.41e-2	2.09e-1	2.33e-1
	2.85e-4	1.66e-3	9.46e-3	3.92e-2	5.03e-2	1.79e-2	6.84e-2	4.35e-2	3.67e-1	4.05e-1

Hu et al. (2024b), retaining its three-layer, width-1024 tanh network and 10,000 training epochs. We use the published adversarial HJB setting for HJB-type equations, with up to 100 sampled training coordinates and 20-step PGD using 10 coordinates. For the remaining semilinear PDEs, we follow the authors' SDE-based setting, using 100 trajectories, 20 time steps, and 10 sampled derivative coordinates.

These baselines retain their source-defined architectures, objectives, and optimization budgets. We substitute the PDE coefficients, terminal conditions, time horizons, and state dimensions required by each benchmark, without introducing Tangent-Actor-specific features or oracle reduced coordinates. Thus, the comparisons are source-faithful transfers to a common whole-function task rather than reproductions of every method's original benchmark.

Table 2: Single-point accuracy across four PDEs (60 method cells). Each method has RMSE above RelMAE; best values are bolded. NF denotes numerical failure of Deep BSDE in the frozen configuration.

Method	Allen–Cahn					Full-Rank Log-Sum-Exp HJB				
Dimension	20	50	100	200	400	20	50	100	200	400
Tangent-Actor	**1.47e-4**	**7.15e-5**	**5.23e-5**	**4.97e-5**	**8.94e-6**	**7.13e-4**	**1.42e-3**	**1.59e-3**	**4.17e-3**	**3.63e-3**
	9.78e-4	**1.08e-3**	**1.19e-3**	**3.49e-3**	**4.94e-4**	**7.84e-4**	**8.76e-4**	**9.04e-4**	**1.33e-3**	**1.25e-3**
FH-MLP	1.04e-3	4.54e-4	2.15e-4	1.16e-4	5.29e-5	8.56e-2	2.71e-1	5.89e-1	1.93e0	3.19e0
	8.13e-3	7.73e-3	7.40e-3	7.86e-3	7.32e-3	1.16e-1	2.03e-1	4.32e-1	7.49e-1	1.24e0
Deep BSDE	3.12e-4	2.22e-4	1.78e-4	1.44e-4	1.09e-4	1.13e-1	2.53e-1	2.82e-1	3.72e-1	3.24e-1
	2.55e-3	3.74e-3	6.50e-3	1.01e-2	1.53e-2	1.58e-1	1.94e-1	2.11e-1	1.49e-1	1.30e-1

Method	Fourier–Tanh Coupled PDE					Joint Nonseparable				
Dimension	20	50	100	200	400	20	50	100	200	400
Tangent-Actor	**4.25e-3**	**4.32e-3**	**3.72e-3**	**8.69e-3**	**1.40e-2**	**1.18e-3**	**1.06e-3**	**1.04e-3**	**1.22e-3**	**1.24e-3**
	6.06e-3	**7.25e-3**	**2.33e-3**	**1.98e-2**	**3.15e-2**	**1.68e-3**	**1.54e-3**	**1.49e-3**	**1.70e-3**	**1.77e-3**
FH-MLP	3.60e-2	4.23e-2	1.33e-1	1.43e-1	2.93e-1	1.14e-2	2.56e-2	5.15e-2	1.02e-1	2.13e-1
	4.98e-2	7.21e-2	1.07e-1	3.56e-1	6.37e-1	1.46e-2	3.44e-2	7.39e-2	1.41e-1	3.00e-1
Deep BSDE	4.38e-2	7.79e-2	1.13e-1	7.52e-2	7.91e-2	2.59e-3	3.76e-3	3.48e-2	*NF*	*NF*
	6.95e-2	1.45e-1	9.37e-2	1.94e-1	1.73e-1	3.35e-3	4.46e-3	1.77e-2	*NF*	*NF*

Evaluation protocol and results. We evaluate eight PDEs covering gradient-only, value-only, and coupled value–gradient nonlinearities at dimensions $d \in \{20, 50, 100, 200, 400\}$. Each method learns a reusable predictor evaluated on the same frozen test states within each setting. Tangent-Actor uses four actor-refinement updates and selects its checkpoint through reference-free validation. All primary whole-function results use one training seed, with ground truth reserved for evaluation.

Table 1 shows that Tangent-Actor achieves the lowest RMSE in 39 of 40 PDE–dimension settings. The exception is the 20D Full-Rank Log-Sum-Exp HJB, where SDGD-PINN performs best. The improvement extends beyond convex or concave drivers: on the 100D oscillatory value–gradient PDE, Tangent-Actor achieves RMSE 8.04×10^{-4}, compared with 1.70×10^{-3} for the strongest baseline. For benchmarks with known low-dimensional structure, that structure is used only to construct independent references; all neural solvers receive the full d-dimensional state.

Solution landscapes. To complement the quantitative results in Table 1, Figure 12 visualizes two-dimensional slices of four high-dimensional PDE solutions. Tangent-Actor closely reproduces the reference value landscapes and achieves the lowest slice-wise value RMSE across all four examples. The improvement is particularly pronounced on the Fourier–Tanh and OU–GMM benchmarks, where the baselines exhibit substantial deviations from the reference solutions. The gradient discrepancy maps provide a more nuanced picture: Tangent-Actor generally captures the gradient structure well, although DPI achieves lower gradient error on the nonconvex coupled PDE. These visualizations illustrate the spatial accuracy underlying the whole-function results.

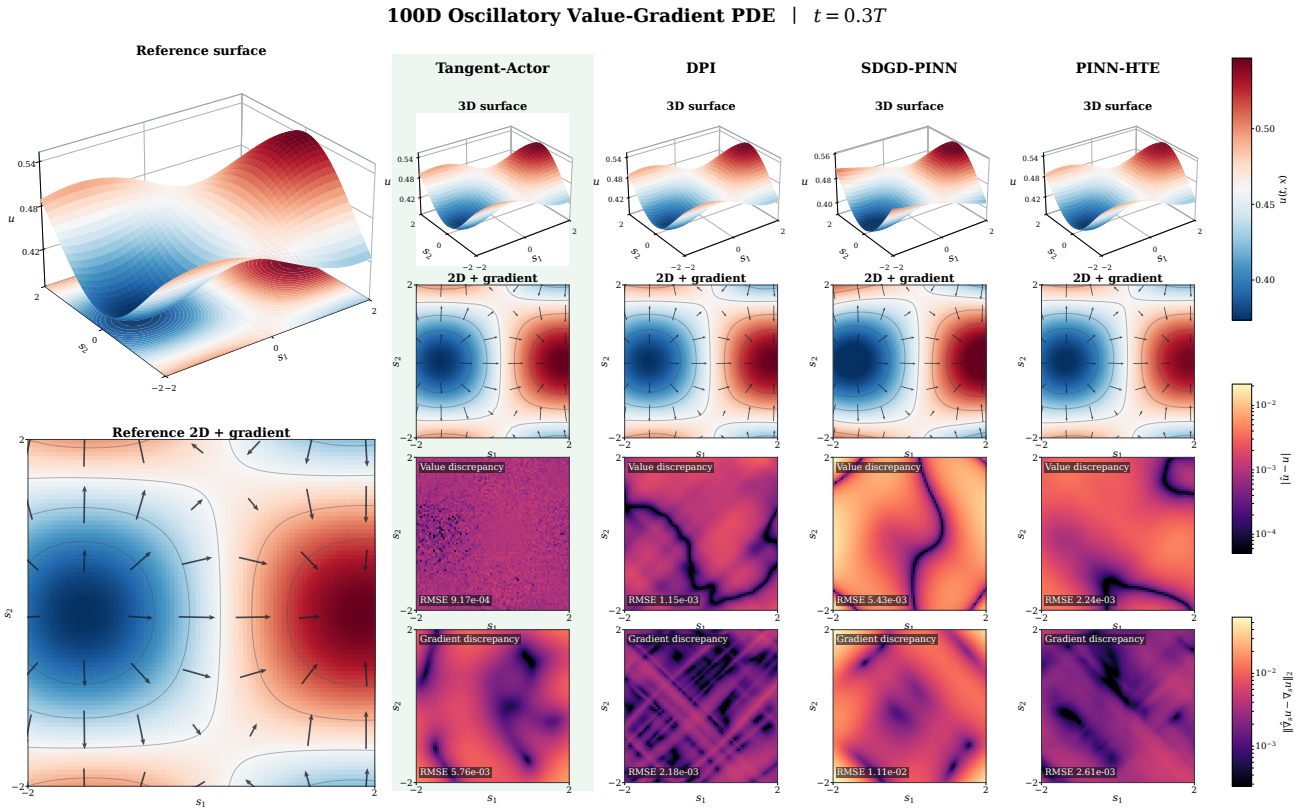


Figure 6: Solution landscapes and errors on four high-dimensional PDEs. Two-dimensional slices at $t = 0.3T$ compare Tangent-Actor with DPI, SDGD-PINN, and PINN-HTE. The reference panels show the solution value, gradient field, and surface. For each method, we show its predicted value, absolute value discrepancy, and projected-gradient discrepancy. Reported RMSE values are evaluated on the displayed slices rather than the whole-function test banks.

5.2 Single-point Accuracy

Baselines and implementation settings. We further compare Tangent-Actor with two established single-point solvers: full-history MLP (FH-MLP) (Hutzenthaler and Kruse, 2020) and Deep BSDE (Han et al., 2018).

For FH-MLP, we implement the standard full-history recursive Monte Carlo estimator, including multilevel telescoping and Bismut-type gradient estimation when required by the nonlinear driver. The primary comparison uses the fixed Picard depth $n = 3$ and branching parameter $M = 16$, with 10 independent Monte Carlo replicates. We retain the unmodified recursive estimator, without value clipping or gradient-weight truncation.

For Deep BSDE, we follow the authors' released single-initial-state implementation. We retain a trainable scalar initial value Y_0, an initial gradient Z_0, and 19 separate time-dependent gradient networks over 20 Euler steps. Each network has two hidden layers of width $d + 10$, with batch normalization. We use the source-family training schedules: 2,000 optimization steps for the HJB-type and coupled problems, and 4,000 for Allen–Cahn. Only the PDE specification and requested initial state are changed. Each query is solved by a separately initialized and trained Deep BSDE model, rather than an amortized predictor.

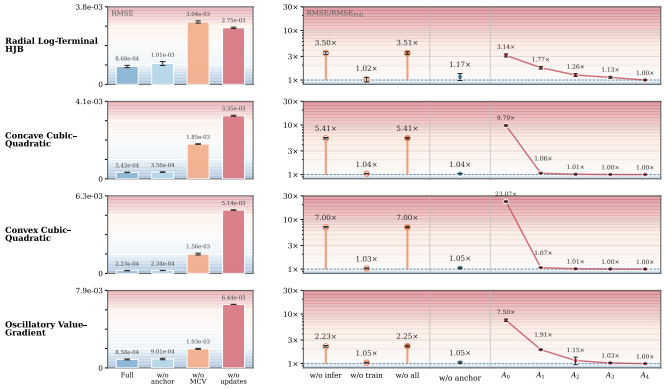


Figure 7: Component ablation across four PDEs.

Evaluation protocol and results. We evaluate 32 fixed time-zero queries per PDE–dimension setting. Tangent-Actor trains one actor per setting and reuses it for all queries, with 1,024 FK–MCV paths and 64 Euler steps at inference.

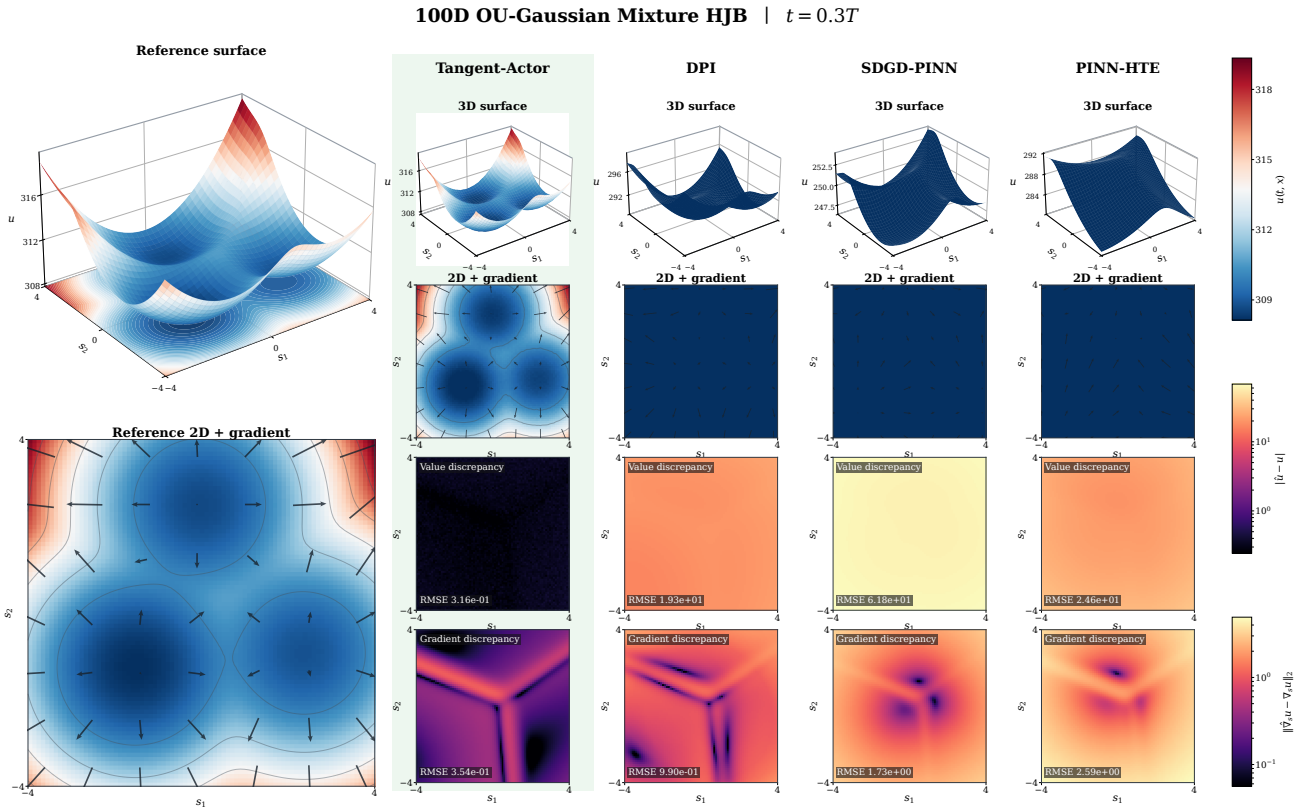


Figure 8: Solution landscapes and errors on four high-dimensional PDEs. Two-dimensional slices at $t = 0.3T$ compare Tangent-Actor with DPI, SDGD-PINN, and PINN-HTE. The reference panels show the solution value, gradient field, and surface. For each method, we show its predicted value, absolute value discrepancy, and projected-gradient discrepancy. Reported RMSE values are evaluated on the displayed slices rather than the whole-function test banks.

FH-MLP independently evaluates each query, whereas Deep BSDE independently optimizes a new model for each initial state. All methods are evaluated against the same reference values.

Table 2 reports the results across four PDEs and five dimensions. Tangent-Actor achieves the lowest RMSE in all 20 PDE–dimension settings. The advantage is particularly pronounced on the Full-Rank Log-Sum-Exp HJB, while Deep BSDE encounters numerical failures on Joint Non-separable at $d \in \{200, 400\}$. These are fixed-configuration accuracy comparisons, rather than matched-compute comparisons.

5.3 Computational Cost and Budget Sensitivity

5.4 Component Ablations

Which ingredients account for the improvements? We ablate martingale control variates (MCV), terminal anchoring, and actor refinement on four 100D PDEs (Figure 7). Removing MCV only at inference increases RMSE by 2.23–7.00×, while removing it only during training changes RMSE by 1.02–1.05×; removing it at both stages closely tracks the inference-only ablation. Replacing the learned tangent by its initial actor A_0 increases RMSE by 3.14–23.07×. Most improvement occurs in the first actor update, with smaller, problem-dependent gains from later iterations. Terminal anchoring has a more modest effect (1.04–1.17×). Thus, the largest observed gains come from learning the tangent and correcting inference-time sampling noise, not from a universally large benefit of every training component.

The accuracy tables do not by themselves establish computational speedups. On the 100D Fourier–Tanh Coupled PDE, H100 training takes approximately 22.6, 10.6, 4.8, and 54.1 minutes for Tangent-Actor, DPI, PINN-HTE, and SDGD-PINN, respectively: Tangent-Actor is not the fastest method in training time. To examine whether increasing baseline budgets closes the accuracy gap, we vary iteration counts, stochastic-estimation resources, and optimization or discretization budgets on two 100D PDEs. The three-seed sweeps in Appendix C show that additional computation sometimes helps but does not improve error monotonically; the tested DPI and PINN-HTE settings do not reach Tangent-Actor's reported accuracy. Deep BSDE is evaluated separately on 64 independently trained queries, rather than the shared whole-function test bank, so that comparison is not paired. These findings support an accuracy advantage under the examined budgets, not universal runtime dominance or

a proven matched-accuracy speedup.

6 Limitations and Scope

The affine tangent construction applies to smooth semilinear drivers without requiring convexity or concavity. The monotone-improvement guarantee in (12) requires joint concavity or convexity on the relevant region; for general drivers, our error bound instead relies on actor consistency and PDE stability. These results do not extend automatically to fully nonlinear PDEs. Monotone improvement assumes exact solutions of the frozen linear PDEs; learned actor updates may not improve monotonically or converge from arbitrary initialization. Pathwise gradient labels require differentiable trajectories and sufficiently regular data, and differentiating through many simulation steps may be memory-intensive. The martingale control variate can increase variance when the actor gradient poorly approximates the gradient of the linearized PDE solution v^A. Neither this correction nor terminal anchoring removes weak Euler bias, approximation error, or optimizer error. The method offers no dimension-free accuracy or complexity guarantee without additional assumptions.

7 Conclusion

We described a high-dimensional semilinear PDE solver built from a reusable tangent actor, linear Feynman–Kac policy evaluation, supervised pathwise gradient updates, and actor-derived martingale control variates. Exact identities relate tangent error to actor mismatch and sampling variance to the difference between the actor and the current policy-value gradient. These results establish a precise target for further analysis and a reproducible algorithmic template, while leaving empirical efficiency against strong native PDE solvers to the completed benchmark study.

Impact Statement

This work concerns numerical methods for PDEs with potential applications in stochastic control and quantitative modeling. Numerical error and uncertified neural convergence may be consequential in downstream decision systems; deployment in high-stakes settings requires independent reference validation, stability checks, and task-specific error controls.

References

Christian Beck, Sebastian Becker, Patrick Cheridito, Arnulf Jentzen, and Ariel Neufeld. Deep splitting method for parabolic pdes. *SIAM Journal on Scientific Computing*, 43(5):A3135–A3154, 2021. doi: 10.1137/19M1297919.

Richard E. Bellman and Robert E. Kalaba. *Quasilinearization and Nonlinear Boundary-Value Problems*. American Elsevier Publishing Company, New York, 1965.

Alain Bensoussan, Jiayue Han, Sheung Chi Phillip Yam, and Xiang Zhou. Value-gradient based formulation of optimal control problem and machine learning algorithm. *SIAM Journal on Numerical Analysis*, 61(2):973–994, 2023. doi: 10.1137/21M1442838.

Jose H. Blanchet and Peter W. Glynn. Unbiased monte carlo for optimization and functions of expectations via multi-level randomization. In *Proceedings of the 2015 Winter Simulation Conference*, pages 3656–3667. IEEE, 2015. doi: 10.1109/WSC.2015.7408524.

Olivier Bokanowski, Stefania Maroso, and Hasnaa Zidani. Some convergence results for howard's algorithm. *SIAM Journal on Numerical Analysis*, 47(4):3001–3026, 2009. doi: 10.1137/08073041X.

Susanne C. Brenner and L. Ridgway Scott. *The Mathematical Theory of Finite Element Methods*. Springer, 3 edition, 2008. doi: 10.1007/978-0-387-75934-0.

Samuel N. Cohen, Jackson Hebner, Deqing Jiang, and Justin Sirignano. Neural actor-critic methods for hamilton-jacobi-bellman pdes: Asymptotic analysis and numerical studies, 2025.

Weinan E, Martin Hutzenthaler, Arnulf Jentzen, and Thomas Kruse. Multilevel picard iterations for solving smooth semilinear parabolic heat equations. *Partial Differential Equations and Applications*, 2(6):80, 2021. doi: 10.1007/s42985-021-00089-5.

Robert Eymard, Thierry Gallouët, and Raphaèle Herbin. Finite volume methods. In *Handbook of Numerical Analysis*, volume 7, pages 713–1018. Elsevier, 2000. doi: 10.1016/S1570-8659(00)07005-8.

Wendell H. Fleming and H. Mete Soner. *Controlled Markov Processes and Viscosity Solutions*. Springer, 2 edition, 2006. doi: 10.1007/0-387-31071-1.

Maximilien Germain, Huyên Pham, and Xavier Warin. Approximation error analysis of some deep backward schemes for nonlinear PDEs. *SIAM Journal on Scientific Computing*, 2022. arXiv:2006.01496.

Emmanuel Gobet, Jean-Philippe Lemor, and Xavier Warin. A regression-based monte carlo method to solve backward stochastic differential equations. *The Annals of Applied Probability*, 15(3):2172–2202, 2005. doi: 10.1214/105051605000000412.

Jiequn Han, Arnulf Jentzen, and Weinan E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018. doi: 10.1073/pnas.1718942115.

Jiequn Han, Wei Hu, Jihao Long, and Yue Zhao. Deep picard iteration for high-dimensional nonlinear pdes. *SIAM Journal on Scientific Computing*, 48(1):C1–C24, 2026. doi: 10.1137/24M169312X.

Pierre Henry-Labordère, Nadia Oudjane, Xiaolu Tan, Nizar Touzi, and Xavier Warin. Branching diffusion representation of semilinear pdes and monte carlo approximation. *Annales de l'Institut Henri Poincaré, Probabilités et Statistiques*, 55(1):184–210, 2019. doi: 10.1214/17-AIHP880.

Zheyuan Hu, Zekun Shi, George Em Karniadakis, and Kenji Kawaguchi. Hutchinson trace estimation for high-dimensional and high-order physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 424:116883, 2024a. doi: 10.1016/j.cma.2024.116883.

Zheyuan Hu, Khemraj Shukla, George Em Karniadakis, and Kenji Kawaguchi. Tackling the curse of dimensionality with physics-informed neural networks. *Neural Networks*, 176:106369, 2024b. doi: 10.1016/j.neunet.2024.106369.

Côme Huré, Huyên Pham, and Xavier Warin. Deep backward schemes for high-dimensional nonlinear pdes. *Mathematics of Computation*, 89(324):1547–1579, 2020. doi: 10.1090/mcom/3514.

Martin Hutzenthaler and Thomas Kruse. Multilevel picard approximations of high-dimensional semilinear parabolic differential equations with gradient-dependent nonlinearities. *SIAM Journal on Numerical Analysis*, 2020. doi: 10.1137/17M1157015.

Martin Hutzenthaler and Tuan Anh Nguyen. Full history recursive multilevel picard approximations suffer from the curse of dimensionality for the hamilton-jacobi-bellman equation of a stochastic control problem, 2025.

Martin Hutzenthaler, Arnulf Jentzen, Thomas Kruse, and Tuan Anh Nguyen. Multilevel picard approximations for high-dimensional semilinear second-order pdes with lipschitz nonlinearities, 2020. URL https://arxiv.org/abs/2009.02484.

Kazufumi Ito, Christoph Reisinger, and Yufei Zhang. A neural network-based policy iteration algorithm with global H^2-superlinear convergence for stochastic games on domains. *Foundations of Computational Mathematics*, 21:331–374, 2021. doi: 10.1007/s10208-020-09460-1.

Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM, 2007. doi: 10.1137/1.9780898717839.

Yiming Meng, Ruikun Zhou, Amartya Mukherjee, Maxwell Fitzsimmons, Christopher Song, and Jun Liu. Physics-informed neural network policy iteration: Algorithms, convergence, and verification. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 35378–35403. PMLR, 2024. URL https://proceedings.mlr.press/v235/meng24b.html.

Huyên Pham. *Continuous-time Stochastic Control and Optimization with Financial Applications*. Springer, 2009. doi: 10.1007/978-3-540-89500-8.

Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

Chang-Han Rhee and Peter W. Glynn. Unbiased estimation with square root convergence for SDE models. *Operations Research*, 63(5):1026–1043, 2015. doi: 10.1287/opre.2015.1404.

Lorenz Richter and Julius Berner. Robust sde-based variational formulations for solving linear pdes via deep learning. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 18649–18666. PMLR, 2022.

Marc Sabate Vidales, David Šiška, and Lukasz Szpruch. Unbiased deep solvers for linear parametric pdes. *Applied Mathematical Finance*, 28(4):299–329, 2022. doi: 10.1080/1350486X.2022.2030773.

Appendix

A Necessary Background from Simple Examples

This appendix introduces the Feynman–Kac representation through two examples. The first explains its connection to Picard iteration and multilevel Monte Carlo. The second derives the stochastic representation used by our Tangent-Actor method.

A.1 Feynman–Kac and Multilevel Picard Methods

Consider the semilinear parabolic PDE

$$\begin{aligned} u_t(t,x) + \frac{1}{2}u_{xx}(t,x) + f(u(t,x)) = 0, \\ u(T,x) = g(x), \end{aligned} \tag{14}$$

where f is a nonlinear function and g is the known terminal condition. For simplicity, assume $f(0) = 0$.

The key idea of Feynman–Kac is to express the solution of a PDE as an expectation over random trajectories. Starting from a fixed point (t, x), consider

$$X_s = x + W_s - W_t, \qquad s \in [t, T], \tag{15}$$

where W_s is standard Brownian motion. Its increments satisfy $W_{s+\Delta s} - W_s \sim \mathcal{N}(0, \Delta s)$.

A useful property of Brownian motion is the shorthand $(dW_s)^2 = ds$. To understand this, observe that each squared increment has expectation $\mathbb{E}[(\Delta W_s)^2] = \Delta s$. Although an individual squared increment is random, the sum of squared increments over an increasingly fine time partition converges to the elapsed time:

$$\sum_i (W_{s_{i+1}} - W_{s_i})^2 \xrightarrow{L^2} T - t. \tag{16}$$

This property, called *quadratic variation*, explains why the second-order term in a stochastic Taylor expansion does not disappear.

Applying Itô's formula to $u(s, X_s)$ gives

$$du(s, X_s) = \left(u_t + \frac{1}{2}u_{xx}\right)(s, X_s)\, ds + u_x(s, X_s)\, dW_s. \tag{17}$$

The additional $\frac{1}{2}u_{xx}$ term comes precisely from the quadratic variation of Brownian motion.

By the PDE, $u_t + \frac{1}{2}u_{xx} = -f(u)$. Integrating from t to T and using $u(T, X_T) = g(X_T)$, we obtain

$$\begin{aligned} u(t,x) = g(X_T) + \int_t^T f(u(s, X_s))\, ds \\ - \int_t^T u_x(s, X_s)\, dW_s. \end{aligned} \tag{18}$$

The last term is an Itô stochastic integral, whose expectation is zero under standard integrability conditions. Taking expectations therefore yields

$$u(t,x) = \mathbb{E}\left[g(X_T) + \int_t^T f(u(s, X_s))\, ds\right]. \tag{19}$$

This is the Feynman–Kac representation. Instead of approximating spatial derivatives on a grid, we can simulate Brownian paths and average their contributions. However, the unknown solution u still appears inside the expectation through $f(u)$. This motivates Picard iteration.

Picard iteration. Picard iteration replaces the unknown solution on the right-hand side of (19) with the approximation from the previous iteration.

Starting from $u_0 = 0$, define

$$u_{n+1}(t,x) = \mathbb{E}\left[g(X_T) + \int_t^T f(u_n(s, X_s))\, ds\right]. \tag{20}$$

Since $f(0) = 0$, the first two iterations are

$$\begin{aligned} u_1(t,x) &= \mathbb{E}[g(X_T)], \\ u_2(t,x) &= \mathbb{E}\left[g(X_T) + \int_t^T f(u_1(s, X_s))\, ds\right]. \end{aligned} \tag{21}$$

The first iteration requires only the known terminal function g. The second evaluates f using the first approximation, and subsequent iterations repeat the same procedure. Under suitable assumptions, these iterates converge to the PDE solution.

The same construction extends to general high-dimensional semilinear parabolic PDEs:

$$\begin{aligned} u_t + \mathcal{L}u + f(t, x, u, \sigma^\top \nabla u) = 0, \\ u(T,x) = g(x), \end{aligned} \tag{22}$$

where $\mathcal{L}u = \mu^\top \nabla u + \frac{1}{2}\operatorname{Tr}(\sigma\sigma^\top \nabla^2 u)$. Here $x \in \mathbb{R}^d$, and the corresponding random trajectories follow

$$dX_s = \mu(s, X_s)\, ds + \sigma(s, X_s)\, dW_s. \tag{23}$$

Applying the same Itô argument gives

$$u(t,x) = \mathbb{E}\Big[g(X_T) + \int_t^T f\big(s, X_s, u(s, X_s), \sigma^\top \nabla u(s, X_s)\big)\, ds\Big]. \tag{24}$$

Thus, the Feynman–Kac principle remains the same: a PDE is represented through expectations along stochastic trajectories, while Picard iteration handles the unknown nonlinear terms.

Monte Carlo and the multilevel idea. The expectations in the Picard iteration can be approximated using Monte Carlo. For example, in our one-dimensional setting, $X_T = x + \sqrt{T-t}\,\xi$ with $\xi \sim \mathcal{N}(0,1)$, so

$$u_1(t,x) \approx \frac{1}{M}\sum_{i=1}^{M} g\left(x + \sqrt{T-t}\,\xi_i\right), \tag{25}$$

where the ξ_i are independent standard Gaussian samples.

For higher iterations, evaluating $u_n(s, X_s)$ requires additional Monte Carlo calculations at random intermediate states. These evaluations become nested as the Picard depth increases.

MLP makes this recursive computation more efficient through the telescoping identity

$$f(u_n) = f(u_0) + \sum_{\ell=1}^{n} \left[f(u_\ell) - f(u_{\ell-1})\right]. \tag{26}$$

As the Picard iterates approach one another, the differences between successive levels typically become smaller. MLP exploits this structure by allocating more Monte Carlo samples to inexpensive lower-level terms and fewer samples to costly higher-level corrections.

A.2 Feynman–Kac for the Tangent-Linearized PDE

We now apply the same Feynman–Kac principle to the linear PDE arising in Tangent-Actor. The key difference is that, once the actor is fixed, all coefficients of the linearized PDE are known functions of time and space.

A simple linear example. Consider the one-dimensional PDE

$$\begin{aligned} v_t + \frac{1}{2}v_{xx} + b(t,x)v_x + c(t,x)v + h(t,x) = 0,\\ v(T,x) = g(x). \end{aligned} \tag{27}$$

Compared with our first example, three known coefficients appear: b modifies the drift, c multiplies the solution, and h is an additional running source term.

Starting from (t,x), define

$$dX_s = b(s,X_s)\,ds + dW_s, \qquad X_t = x. \tag{28}$$

Unlike standard Brownian motion, this process has a drift determined by b.

We also introduce the weight

$$C_s = \exp\left(\int_t^s c(r,X_r)\,dr\right). \tag{29}$$

This weight accounts for the term cv in the PDE.

Applying Itô's formula and the product rule to $C_s v(s,X_s)$ gives

$$d(C_s v(s,X_s)) = -C_s h(s,X_s)\,ds + C_s v_x(s,X_s)\,dW_s. \tag{30}$$

Here, the terms involving v_t, v_{xx}, bv_x, and cv cancel by (27).

Integrating to T and taking expectations eliminates the stochastic integral, yielding

$$v(t,x) = \mathbb{E}\left[C_T g(X_T) + \int_t^T C_s h(s,X_s)\,ds\right]. \tag{31}$$

This formula is especially useful: the expectation involves only the terminal condition and the known coefficients. We can therefore evaluate $v(t,x)$ by simulating trajectories, without recursively evaluating an unknown solution inside the expectation.

Application to Tangent-Actor. In our method, freezing an actor A replaces the nonlinear driver with

$$\mathcal{T}_A(y,z) = c_A y + b_A^\top z + h_A.$$

The corresponding high-dimensional linear PDE is

$$\begin{aligned} v_t^A + \mathcal{L}v^A + c_A v^A + b_A^\top \sigma^\top \nabla v^A + h_A = 0,\\ v^A(T,x) = g(x). \end{aligned} \tag{32}$$

The same derivation applies with the actor-dependent stochastic process

$$dX_s^A = [\mu + \sigma b_A](s,X_s^A)\,ds + \sigma(s,X_s^A)\,dW_s, X_t^A = x. \tag{33}$$

Writing

$$C_s^A = \exp\left(\int_t^s c_A(r,X_r^A)\,dr\right),$$

we obtain

$$v^A(t,x) = \mathbb{E}\left[C_T^A g(X_T^A) + \int_t^T C_s^A h_A(s,X_s^A)\,ds\right]. \tag{34}$$

This is precisely the Feynman–Kac representation used in Section 3. The actor determines the coefficients of a linear PDE, and Monte Carlo estimates its solution.

Martingale control variates. The same Itô calculation also explains our variance-reduction step. Denote the random payoff in (34) by P^A. Before taking expectations, the stochastic identity gives

$$P^A = v^A(t,x) + \int_t^T C_s^A z_{v^A}(s,X_s^A)^\top\,dW_s, \tag{35}$$

where $z_{v^A} = \sigma^\top \nabla v^A$.

Thus, fluctuations of P^A around its expectation are described by a stochastic integral involving the solution gradient.

Tangent-Actor already learns a gradient approximation q. We use it to construct

$$\widetilde{P}^A = P^A - \int_t^T C_s^A q(s, X_s^A)^\top \, dW_s. \tag{36}$$

Since the correction has zero expectation under standard integrability conditions,

$$\mathbb{E}[\widetilde{P}^A] = \mathbb{E}[P^A] = v^A(t,x).$$

When q approximates z_{v^A} well, the correction removes much of the random fluctuation in the original payoff. The corresponding variance identity and theoretical guarantees are developed in Section 4.

In summary, Feynman–Kac transforms PDEs into stochastic expectations, Picard iteration resolves the nonlinear dependence on the unknown solution, and multilevel sampling reduces the computational cost of the resulting recursion.

B Proofs and Technical Assumptions

This appendix develops the statistical and PDE identities underlying Section 4. We first establish the error introduced by nonlinear evaluation of Monte Carlo estimates, then relate tangent consistency to the PDE solution. We next characterize exact actor improvement for concave drivers and the variance of martingale-corrected value and gradient targets.

B.1 Setting and standing assumptions

Let W be an m-dimensional Brownian motion and let $u : [0,T] \times \mathbb{R}^d \to \mathbb{R}$ solve

$$\begin{aligned} &\partial_t u + \mathcal{L}u + f(t,x,u,z_u) = 0, \\ &z_u = \sigma^\top \nabla u, \qquad u(T,\cdot) = g. \end{aligned} \tag{37}$$

where $\mathcal{L}\phi = \mu^\top \nabla\phi + \frac{1}{2}\operatorname{Tr}(\sigma\sigma^\top \nabla^2 \phi)$. We abbreviate $Y_w = (w, z_w) \in \mathbb{R}^{1+m}$ for a smooth function w. For a fixed, state-dependent actor $A = (r, q)$, let

$$\begin{aligned} \mathcal{T}_A(t,x;y,z) &= f(t,x,A) + Df(t,x,A)^\top [(y,z) - A] \\ &= c_A y + b_A^\top z + h_A, \end{aligned} \tag{38}$$

with $c_A = f_y(t,x,A)$, $b_A = \nabla_z f(t,x,A)$, and $h_A = f(t,x,A) - c_A r - b_A^\top q$. The associated linear PDE is

$$\partial_t v^A + \mathcal{L}v^A + \mathcal{T}_A(t,x;Y_{v^A}) = 0, \qquad v^A(T,\cdot) = g. \tag{39}$$

We work under the following standard conditions, tailored to the identities being used.

(H1) PDE and diffusion regularity. The coefficients μ, σ and the terminal data g have the regularity and growth needed for the corresponding diffusion and Feynman–Kac formulas. Equation (37) has a unique classical solution u, and each considered actor A admits a classical solution v^A of (39). The relevant solution values and tangent points lie in a common convex domain $\mathcal{D} \subseteq \mathbb{R}^{1+m}$.

(H2) Smooth driver and stable linear equations. For every (t,x), the map $Y \mapsto f(t,x,Y)$ is continuously differentiable on $\mathcal{D}$. Its value derivative satisfies $|f_y(t,x,Y)| \le L_y$ for some $L_y \ge 0$. Where second-order estimates are invoked, f is twice differentiable and

$$\|D_Y^2 f(t,x,Y)\|_{\mathrm{op}} \le H \tag{40}$$

on every segment between the relevant states. The diffusions with drifts $\mu + \sigma b_A$ and with the interpolated gradients of f appearing below are well posed. Their discount factors and the stochastic integrals in the proofs satisfy the required moment and martingale conditions.

(H3) Pathwise differentiation. For results involving gradient supervision, the controlled stochastic flow, discounted payoff, and martingale correction are differentiable with respect to the initial state in L^2. Their derivatives may be interchanged with expectations, time integrals, and Itô integrals. Smooth diffusion and actor coefficients with suitable derivative and moment bounds provide sufficient conditions.

Each theorem treats the actor as fixed. When its parameters are obtained by stochastic training, expectations are understood conditionally on the trained actor, using fresh Brownian samples for its evaluation.

B.2 Nonlinear Monte Carlo Bias

Consider a fixed state and a noisy gradient estimate $\widehat{z} = z + b + \varepsilon$, where $b = \mathbb{E}[\widehat{z}] - z$, $\mathbb{E}[\varepsilon] = 0$, and $\Sigma = \operatorname{Cov}(\varepsilon)$. Expanding the squared norm gives

$$\mathbb{E}\|\widehat{z}\|^2 - \|z\|^2 = 2z^\top b + \|b\|^2 + \operatorname{tr}(\Sigma). \tag{41}$$

Thus, even when $\widehat{z}$ is unbiased ($b = 0$), a quadratic driver introduces a plug-in bias of $\operatorname{tr}(\Sigma)$. For an average of M independent gradient estimates with per-sample covariance Σ_0, this becomes $\operatorname{tr}(\Sigma_0)/M$, scaling as d/M when $m = d$ and coordinate variances are of constant order. For the driver $f(z) = -\frac{1}{2}\|z\|^2$, the bias has the corresponding negative sign and factor $1/2$.

B.3 Tangent consistency and nonlinear PDE accuracy

Let $Y^A := Y_{v^A} = (v^A, z_{v^A})$, and define the nonlinear residual of the fixed-actor value by

$$\begin{aligned} R_A &= \partial_t v^A + \mathcal{L}v^A + f(t,x,Y^A) \\ &= f(t,x,Y^A) - \mathcal{T}_A(t,x;Y^A). \end{aligned} \tag{42}$$

The second equality follows directly from the linear equation (39).

Proposition B.1 (Second-order tangent consistency)**.** *Under (H1)–(H2), let $\delta_A = Y^A - A$ and assume the Hessian bound* (40) *holds along $A + \theta\delta_A$ for $\theta \in [0,1]$. Then*

$$R_A(t,x) = \int_0^1 (1-\theta)\, \delta_A^\top D_Y^2 f(t,x,A+\theta\delta_A)\delta_A \, d\theta, \tag{43}$$

and consequently

$$|R_A(t,x)| \le \frac{H}{2}\|Y^A(t,x) - A(t,x)\|^2. \tag{44}$$

If $A = Y^A$ everywhere, then $v^A = u$.

Proof. Apply the second-order Taylor formula with integral remainder to the function $Y \mapsto f(t,x,Y)$, expanding at A and evaluating at $Y^A = A + \delta_A$:

$$f(t,x,Y^A) = \mathcal{T}_A(t,x;Y^A) + \int_0^1 (1-\theta)\delta_A^\top H_\theta \delta_A \, d\theta,$$

where $H_\theta = D_Y^2 f(t,x,A+\theta\delta_A)$. The operator-norm bound on the Hessian and $\int_0^1(1-\theta)\,d\theta = 1/2$ give (44). At an exact actor fixed point, $\delta_A = 0$, so $R_A = 0$. Equation (42) and the shared terminal condition show that v^A solves the nonlinear PDE. Uniqueness identifies it with u. □

The preceding estimate supports a reference-free consistency criterion. For a distribution ν over training or validation states, define

$$\mathcal{S}_\nu(A) = \mathbb{E}_{(t,x)\sim\nu}\|Y^A(t,x) - A(t,x)\|^2.$$

Integrating (44) yields the useful population relation

$$\mathbb{E}_\nu|R_A| \le \frac{H}{2}\mathcal{S}_\nu(A). \tag{45}$$

It links actor self-consistency to the nonlinear residual on the same state distribution.

To relate the residual to the error of the PDE solution, we use a linear equation for the difference $v^A - u$.

Proposition B.2 (Residual-to-solution stability)**.** *Assume (H1)–(H2), including well-posedness of the diffusion with the interpolated drift constructed below, and suppose R_A and $Y^A - A$ have finite sup norms. Then*

$$\begin{aligned}\|v^A - u\|_\infty &\le Te^{L_y T}\|R_A\|_\infty \\ &\le \frac{HTe^{L_y T}}{2}\|Y^A - A\|_\infty^2.\end{aligned} \tag{46}$$

More precisely, the error is bounded by a weighted expectation of $\|Y^A - A\|^2$ along a diffusion associated with the interpolated gradient of f.

Proof. Set $w = v^A - u$ and introduce the segment $Y_\theta = Y_u + \theta(Y^A - Y_u)$. By the fundamental theorem of calculus,

$$f(t,x,Y^A) - f(t,x,Y_u) = \alpha(t,x)w + \beta(t,x)^\top z_w,$$

where $z_w = \sigma^\top \nabla w$ and

$$\alpha(t,x) = \int_0^1 f_y(t,x,Y_\theta)\,d\theta,$$
$$\beta(t,x) = \int_0^1 \nabla_z f(t,x,Y_\theta)\,d\theta.$$

Subtract the nonlinear equation for u from the residual equation for v^A in (42):

$$\partial_t w + \mathcal{L}w + \beta^\top z_w + \alpha w = R_A, \qquad w(T,\cdot) = 0. \tag{47}$$

Let X^β solve

$$dX_s^\beta = [\mu + \sigma\beta](s,X_s^\beta)\,ds + \sigma(s,X_s^\beta)\,dW_s, \qquad X_t^\beta = x,$$

and let $D_s^\beta = \exp(\int_t^s \alpha(\ell, X_\ell^\beta)\,d\ell)$. Applying Feynman–Kac to (47) gives

$$w(t,x) = -\mathbb{E}_{t,x}^\beta \int_t^T D_s^\beta R_A(s,X_s^\beta)\,ds. \tag{48}$$

Since $|\alpha| \le L_y$, the discount factor satisfies $0 < D_s^\beta \le e^{L_y(s-t)}$. Using (44) under the expectation,

$$|w(t,x)| \le \frac{H}{2}\mathbb{E}_{t,x}^\beta \int_t^T e^{L_y(s-t)}\|Y^A - A\|^2(s,X_s^\beta)\,ds.$$

Taking suprema yields (46). □

Noise-corrected reference-free scores. Suppose $\widehat{Y}^A(t,x)$ is a stochastic value–gradient label with conditional mean $Y^A(t,x)$ and conditional covariance $\Sigma_A(t,x)$. For any fixed actor A,

$$\mathbb{E}\big[\|\widehat{Y}^A - A\|^2 \mid t,x,A\big] = \|Y^A - A\|^2 + \operatorname{tr}\Sigma_A. \tag{49}$$

Independent label replications therefore allow the variance term to be estimated and subtracted, yielding an unbiased estimator of the pointwise squared actor-consistency error for the specified linear-evaluation target. For value and gradient terms with different loss weights, the same identity holds in a weighted Euclidean norm, with $\operatorname{tr}(W\Sigma_A)$ replacing the covariance trace. Equation (45) then connects population consistency to the average residual.

B.4 Concave drivers and exact actor improvement

Here assume additionally that $f(t,x,\cdot)$ is jointly concave on $\mathcal{D}$, which contains A, Y_u, and all $Y_{v^{A_j}}$ considered. Define the tangent gap

$$D_A(t,x;Y) = \mathcal{T}_A(t,x;Y) - f(t,x,Y) \ge 0. \tag{50}$$

We use $D_A(Y_w)(t,x)$ as shorthand for $D_A(t,x;Y_w(t,x))$. This gap measures the difference between a chosen tangent and the nonlinear driver at a given value–gradient pair.

Proposition B.3 (Tangent-regret identity)**.** *For any fixed actor A satisfying (H1)–(H2),*

$$\boxed{v^A(t,x) - u(t,x) = \mathbb{E}_{t,x}^A \int_t^T C_s^A \, D_A(Y_u)(s,X_s^A)\,ds.} \tag{51}$$

In particular, $v^A \geq u$ pointwise. If the Hessian bound (40) *holds along the segment between A and Y_u, then*

$$0 \leq v^A(t,x) - u(t,x) \leq \frac{H}{2}\mathbb{E}^A_{t,x}\int_t^T C^A_s \|Y_u - A\|^2(s, X^A_s)\, ds.$$

Proof. Write $\mathcal{L}^A = \mathcal{L} + b_A^\top \sigma^\top \nabla$. The true solution satisfies

$$\partial_t u + \mathcal{L}^A u + c_A u + h_A = -f(t,x,Y_u) + \mathcal{T}_A(t,x;Y_u) = D_A(t,x;Y_u).$$

In contrast, v^A solves $\partial_t v^A + \mathcal{L}^A v^A + c_A v^A + h_A = 0$. Their difference $w = v^A - u$ consequently satisfies

$$\partial_t w + \mathcal{L}^A w + c_A w = -D_A(t,x;Y_u), \qquad w(T,\cdot) = 0.$$

Feynman–Kac yields (51). Concavity gives $D_A \geq 0$, while the discount weight C^A_s is strictly positive. The second-order tangent remainder establishes the final bound. □

For $f(z) = -\frac{1}{2}\|z\|^2$, take $A = q = a$. Its tangent gap is exactly $D_a(z) = \frac{1}{2}\|a - z\|^2$, and $c_a = 0$. In the illustrative case $\mu = 0$ and $\sigma = \sqrt{2}I_d$,

$$\boxed{v^a(t,x) - u(t,x) = \frac{1}{2}\mathbb{E}^a_{t,x}\int_t^T \|a - z_u\|^2(s, X^a_s)\, ds.} \tag{52}$$

where $dX^a_s = -\sqrt{2}a(s, X^a_s)\, ds + \sqrt{2}\, dW_s$.

Proposition B.4 (Exact and verifiable actor improvement)**.** *Let $v_j = v^{A_j}$ and $Y_j = (v_j, z_{v_j})$. For any next actor A_{j+1} for which the linear evaluation is well posed, write $v_{j+1} = v^{A_{j+1}}$. Then*

$$v_j(t,x) - v_{j+1}(t,x) = \mathbb{E}^{A_{j+1}}_{t,x}\int_t^T C^{A_{j+1}}_s \Delta_j(s, X^{A_{j+1}}_s)\, ds, \tag{53}$$

where

$$\Delta_j(t,x) = \mathcal{T}_{A_j}(t,x;Y_j) - \mathcal{T}_{A_{j+1}}(t,x;Y_j).$$

For the exact tangent update $A_{j+1} = Y_j$, $\Delta_j = D_{A_j}(Y_j) \geq 0$, and hence

$$\boxed{u(t,x) \leq v_{j+1}(t,x) \leq v_j(t,x).} \tag{54}$$

The same ordering holds for any update satisfying $\Delta_j(t,x) \geq 0$ throughout the domain.

Proof. The preceding actor v_j satisfies

$$\partial_t v_j + \mathcal{L} v_j + \mathcal{T}_{A_j}(t,x;Y_j) = 0.$$

Evaluate the new actor's affine driver at Y_j:

$$\partial_t v_j + \mathcal{L} v_j + \mathcal{T}_{A_{j+1}}(t,x;Y_j) = -\big(\mathcal{T}_{A_j}(t,x;Y_j) - \mathcal{T}_{A_{j+1}}(t,x;Y_j)\big) = -\Delta_j.$$

Subtract the linear PDE for $v_{j+1} = v^{A_{j+1}}$. The difference $w_j = v_j - v_{j+1}$ solves

$$\partial_t w_j + \mathcal{L}^{A_{j+1}} w_j + c_{A_{j+1}} w_j = -\Delta_j, \qquad w_j(T,\cdot) = 0.$$

Applying Feynman–Kac gives (53). For $A_{j+1} = Y_j$, the tangent identity $\mathcal{T}_{Y_j}(Y_j) = f(Y_j)$ yields $\Delta_j = D_{A_j}(Y_j)$. Its nonnegativity follows from concavity. Finally, Proposition B.3 gives $v_{j+1} \geq u$ and establishes the ordering. □

Convex counterpart. For a jointly convex driver, the tangent inequality reverses: $\mathcal{T}_A(Y) \leq f(Y)$. The same two Feynman–Kac identities then yield $v^A \leq u$ and, for the exact tangent update, $v_j \leq v_{j+1} \leq u$. The corresponding result is obtained by applying the arguments above to the gap $f(Y) - \mathcal{T}_A(Y) \geq 0$.

Corollary B.5 (Identification of a regular limiting actor)**.** *Under the exact updates $A_{j+1} = Y_j$, suppose that v_j and the derivatives entering its PDE converge locally uniformly, that the terminal condition is preserved in the limit, and that the resulting limit can be substituted in the PDE. Then v_j converges to the unique nonlinear solution u.*

Proof. Write $v_j \to v_\infty$. The regularity hypotheses give $A_{j+1} = Y_{v_j} \to Y_{v_\infty}$, and hence $A_j \to Y_{v_\infty}$ as well. Passing to the limit in (39) produces

$$\partial_t v_\infty + \mathcal{L} v_\infty + \mathcal{T}_{Y_{v_\infty}}(t,x;Y_{v_\infty}) = 0.$$

The tangent agrees with f at its base point. Therefore v_∞ solves (37) with the terminal condition g, and uniqueness gives $v_\infty = u$. □

B.5 Martingale correction and pathwise gradient labels

For a fixed actor A, introduce the controlled diffusion

$$dX^A_s = B_A(s, X^A_s)\, ds + \sigma(s, X^A_s)\, dW_s, \qquad X^A_t = x, \qquad B_A = \mu + \sigma b_A.$$

and the positive weight $C^A_s = \exp(\int_t^s c_A(\ell, X^A_\ell)\, d\ell)$. The linear Feynman–Kac payoff is

$$P^A = C^A_T g(X^A_T) + \int_t^T C^A_s h_A(s, X^A_s)\, ds.$$

Proposition B.6 (Martingale representation and MCV variance)**.** *Under (H1)–(H2), with the stated square-integrability conditions,*

$$P^A - v^A(t,x) = \int_t^T C^A_s z_{v^A}(s, X^A_s)^\top\, dW_s. \tag{55}$$

For any predictable, square-integrable gradient actor q, let

$$H_q^A = \int_t^T C_s^A q(s, X_s^A)^\top \, dW_s, \qquad \widetilde{P}^A = P^A - H_q^A.$$

Then

$$\mathbb{E}_{t,x}[\widetilde{P}^A] = v^A(t,x) \tag{56}$$

and

$$\boxed{\mathrm{Var}_{t,x}(\widetilde{P}^A) = \mathbb{E}_{t,x}^A \int_t^T (C_s^A)^2 \, \|z_{v^A} - q\|^2(s, X_s^A)\, ds.} \tag{57}$$

For the average of M conditionally independent corrected payoffs, the variance equals the right-hand side divided by M.

Proof. Apply Itô's product rule to $C_s^A v^A(s, X_s^A)$ and substitute the frozen linear PDE:

$$d(C_s^A v^A(s, X_s^A)) = -C_s^A h_A(s, X_s^A)\, ds + C_s^A z_{v^A}(s, X_s^A)^\top \, dW_s.$$

Integrating from t to T and using $v^A(T, \cdot) = g$ gives (55). Subtract the correction H_q^A to obtain

$$\widetilde{P}^A - v^A(t,x) = \int_t^T C_s^A (z_{v^A} - q)(s, X_s^A)^\top \, dW_s.$$

The Itô integral has conditional mean zero, giving (56). Applying Itô isometry yields (57); independence of the M paths gives the sample-mean variance formula. □

Equation (57) measures the quality of the control variate in a trajectory-weighted gradient norm. When $q = z_{v^A}$ as a function along the controlled paths, the corrected payoff is constant and both the continuous-time Value variance and, under (H3), its pathwise Gradient variance vanish.

Proposition B.7 (Differentiation of corrected payoffs)**.** *Assume (H3), and couple all initial states using the same Brownian motion. At each fixed (t, x),*

$$\boxed{\mathbb{E}_{t,x}\big[\sigma(t,x)^\top \nabla_x \widetilde{P}^A(t, x; W)\big] = z_{v^A}(t,x).} \tag{58}$$

Moreover, put

$$G_s(x) = C_s^{A,x}\big(z_{v^A} - q\big)(s, X_s^{A,x}) \in \mathbb{R}^m.$$

Let $J_s^G(x) = \nabla_x G_s(x) \in \mathbb{R}^{m \times d}$. For the single-path gradient estimator $Z^A(t, x; W) = \sigma(t,x)^\top \nabla_x \widetilde{P}^A(t, x; W)$,

$$\mathbb{E}_{t,x}\|Z^A(t,x;W) - z_{v^A}(t,x)\|^2 = \mathbb{E}_{t,x} \int_t^T \big\|\sigma(t,x)^\top [J_s^G(x)]^\top\big\|_F^2 \, ds. \tag{59}$$

Consequently, the corresponding M-path average has gradient mean-squared error equal to the right-hand side divided by M.

Proof. By (56), the corrected payoff has expectation $v^A(t,x)$ for each initial point x. Assumption (H3) permits differentiation under the expectation:

$$\mathbb{E}[\sigma(t,x)^\top \nabla_x \widetilde{P}^A] = \sigma(t,x)^\top \nabla_x \mathbb{E}[\widetilde{P}^A] = \sigma(t,x)^\top \nabla_x v^A = z_{v^A}(t,x).$$

For the variance identity, write the corrected martingale representation as

$$\widetilde{P}^A(t,x;W) - v^A(t,x) = \int_t^T G_s(x)^\top \, dW_s.$$

Differentiating this identity in L^2 gives

$$\nabla_x \widetilde{P}^A(t,x;W) - \nabla_x v^A(t,x) = \int_t^T [J_s^G(x)]^\top \, dW_s.$$

Multiply by the fixed matrix $\sigma(t,x)^\top$ and apply the vector-valued Itô isometry to obtain (59). □

For clarity about state dependence, set $B_A = \mu + \sigma b_A$ and $J_s = \partial X_s^{A,x}/\partial x$. Under the smoothness conditions in (H3), $J_t = I_d$ and

$$dJ_s = (\nabla_x B_A)(s, X_s^A) J_s \, ds + \sum_{k=1}^m (\nabla_x \sigma_{\cdot k})(s, X_s^A) J_s \, dW_s^k. \tag{60}$$

Thus both the forward drift and diffusion contribute to the initial-state derivative. The state Jacobians of c_A, h_A, and q likewise appear when differentiating the weight, running payoff, and martingale correction. Freezing network parameters preserves all of these input–state derivatives.

B.6 Euler discretization and noisy supervised learning

Let $t = t_0 < t_1 < \cdots < t_N = T$, with increments $\Delta t_n = t_{n+1} - t_n$ and $\Delta W_n = W_{t_{n+1}} - W_{t_n}$. A left-endpoint Euler approximation of the controlled diffusion and weight is

$$\begin{aligned} X_{n+1} &= X_n + B_A(t_n, X_n)\Delta t_n + \sigma(t_n, X_n)\Delta W_n, \\ C_{n+1} &= C_n \exp\big(c_A(t_n, X_n)\Delta t_n\big), \qquad C_0 = 1. \end{aligned} \tag{61}$$

The resulting payoff and control variate are

$$\begin{aligned} P_N^A &= C_N g(X_N) + \sum_{n=0}^{N-1} C_n h_A(t_n, X_n)\Delta t_n, \\ H_{q,N}^A &= \sum_{n=0}^{N-1} C_n q(t_n, X_n)^\top \Delta W_n, \\ \widetilde{P}_N^A &= P_N^A - H_{q,N}^A. \end{aligned} \tag{62}$$

Proposition B.8 (Finite-step centering and error decomposition)**.** *Fix A and suppose the Euler payoffs have finite second moments. Let $v_N^A(t,x) = \mathbb{E}[P_N^A \mid A]$. Then*

$$\mathbb{E}[H_{q,N}^A \mid A] = 0, \qquad \mathbb{E}[\widetilde{P}_N^A \mid A] = v_N^A(t,x). \tag{63}$$

For M conditionally independent discretized trajectories, set $\widehat{v}_{M,N}^A = M^{-1}\sum_i \widetilde{P}_{N,i}^A$. Then

$$\begin{aligned}&\mathbb{E}\big[(\widehat{v}_{M,N}^A - u(t,x))^2 \mid A\big] \\ &\quad = (v_N^A(t,x) - u(t,x))^2 + \frac{1}{M}\operatorname{Var}(\widetilde{P}_N^A \mid A).\end{aligned} \tag{64}$$

When the finite-step payoff is pathwise differentiable and interchange is valid, the derivative of its corrected average is unbiased for $\sigma(t,x)^\top \nabla_x v_N^A(t,x)$.

Proof. At every time step, $C_n q(t_n, X_n)$ is measurable with respect to the information available at t_n, while $\mathbb{E}[\Delta W_n \mid \mathcal{F}_{t_n}] = 0$. Thus

$$\mathbb{E}[C_n q(t_n, X_n)^\top \Delta W_n \mid A] = 0,$$

and summing over n gives (63). The corrected sample mean has conditional expectation v_N^A and conditional variance $\operatorname{Var}(\widetilde{P}_N^A \mid A)/M$. Expanding its squared error around v_N^A gives (64), because the cross term has zero conditional expectation. Differentiating (63) with respect to x under the stated conditions establishes the gradient claim. □

For antithetic simulation, the independent sampling units are Brownian-path *pairs*; the same decomposition applies to the average of independent pair payoffs. The first term in (64) combines actor linearization and time-discretization errors, while the second measures finite-sample uncertainty. Time-step refinement and additional paths control these two contributions independently.

Finally, consider the learning objective. For a fixed training state and frozen source actor, let $\widehat{Y} = Y^A + \epsilon$ with $\mathbb{E}[\epsilon \mid t, x, A] = 0$. For any fixed candidate output $B_\theta(t,x)$ and any positive semidefinite weight matrix W,

$$\begin{aligned}&\mathbb{E}\big[\|B_\theta - \widehat{Y}\|_W^2 \mid t, x, A\big] \\ &\quad = \|B_\theta - Y^A\|_W^2 + \mathbb{E}\big[\|\epsilon\|_W^2 \mid t, x, A\big],\end{aligned} \tag{65}$$

where $\|v\|_W^2 = v^\top W v$. For discretized labels, replace Y^A by the corresponding finite-step value–gradient target. This identity shows that conditionally centered stochastic supervision preserves the population regression target, while variance reduction decreases the additive noise component of the training risk.

B.7 Terminal-aware actor construction

Terminal anchoring uses the known value g and, for smooth terminal data, its gradient. Put $\tau = T - t$. For an unrestricted Value head, the parameterization

$$\begin{aligned}r_\theta(t,x) &= g(x) + \tau R_\theta(t,x), \\ q_\theta(t,x) &= \sigma(T,x)^\top \nabla g(x) + \tau Q_\theta(t,x)\end{aligned} \tag{66}$$

satisfies both terminal identities exactly, for any neural functions R_θ, Q_θ.

If the PDE admits a certified Value interval (r_-, r_+) and the terminal data satisfy $r_- < g(x) < r_+$, let

$$p_g(x) = \frac{g(x) - r_-}{r_+ - r_-} \in (0,1).$$

A range-preserving choice is

$$\begin{aligned}r_\theta(t,x) = r_- + (r_+ - r_-)& \\ \times \operatorname{sigmoid}(\operatorname{logit} p_g(x) + \tau R_\theta(t,x))&.\end{aligned} \tag{67}$$

At $t = T$, the factor τ vanishes and $\operatorname{sigmoid}(\operatorname{logit} p_g) = p_g$, so $r_\theta(T,x) = g(x)$. For every (t,x), the sigmoid range gives $r_- < r_\theta(t,x) < r_+$. The Gradient head in (66) likewise satisfies $q_\theta(T,x) = \sigma(T,x)^\top \nabla g(x)$. These properties encode available terminal and invariant-domain information directly into the approximation class.

Together, the results above quantify three complementary components of Tangent-Actor: the quadratic residual of tangent approximation, the optimization structure of exact actor updates, and the martingale variance of stochastic evaluation. They provide a common mathematical basis for actor-consistency validation, terminal-aware supervision, and numerical refinement.

C Sensitivity to Increased Computational Budgets

Motivation and experimental design. In the original 100-dimensional *Fourier–Tanh Coupled PDE* experiment, Tangent-Actor, DPI, PINN–HTE, and SDGD-PINN require approximately 22.6, 10.6, 4.8, and 54.1 minutes of training, respectively. To test whether the lower budgets of DPI and PINN–HTE explain their accuracy gaps, we study increased computation on two 100-dimensional problems: the *Full-Rank Log-Sum-Exp HJB* and the *Fourier–Tanh Coupled PDE*. Deep BSDE is examined separately as a query-wise solver; SDGD-PINN is not scaled further in this study. For each method, one computational hyperparameter is varied at a time, with all other training and model settings fixed. Table 3 reports the first three configurations in increasing parameter order from each frozen sweep, not the configurations with the smallest test errors. All budget sweeps use NVIDIA H100 GPUs and three independent training seeds.

Budget axes. The three sweeps address distinct sources of computation. **C0** increases nonlinear or optimization iterations: Picard rounds for DPI, training epochs for PINN–HTE, and optimizer updates for Deep BSDE. **C1** increases stochastic-estimation resources: integral Monte Carlo samples for DPI, Hutchinson probes for PINN–HTE, and Brownian paths per optimization step (batch size) for Deep BSDE. **C2** increases epochs per Picard iteration for DPI, training batch size for PINN–HTE, and Euler time-discretization

steps for Deep BSDE. The last choice also changes discretization error; C2 is therefore not a common optimizer-budget axis across algorithms. C0 entries are genuine checkpoints from continuous training trajectories; other nonidentical configurations are trained separately. The common Deep BSDE reference configuration is reused across sweeps where its numerical settings coincide.

Results and interpretation. Increasing C0 reduces Deep BSDE's mean RMSE from 0.1648 to 0.1447 on the Full-Rank Log-Sum-Exp HJB and from 0.0767 to 0.0595 on the Fourier–Tanh Coupled PDE, at cumulative 64-query training costs of 85.8 and 89.9 minutes for the final settings, respectively. Increasing the Deep BSDE batch size gives smaller improvements, while increasing its Euler steps from 10 to 40 does not improve accuracy at the tested fixed optimizer budget. DPI and PINN–HTE likewise display nonmonotone responses to additional computation; for instance, increased Picard rounds improve DPI on the Fourier–Tanh problem but not on the Full-Rank HJB over these three displayed settings. The smallest observed errors remain well above the historical Tangent-Actor reference RMSEs of approximately 7.20×10^{-4} and 1.76×10^{-3}, respectively, on the 960-state banks. For Deep BSDE, this is a descriptive rather than paired comparison because its RMSE is measured on a 64-state subset. Thus, the tested single-parameter increases do not close the observed gap; these finite-budget observations cannot rule out improvements from joint tuning, different schedules, or larger budgets. No joint-scaling sweep is included.

Raw component-ablation results. The normalized effects in the component-ablation figure are computed from the absolute whole-function value errors reported in Table 4. Every entry uses the same frozen 960-state test bank, 64 Euler steps, and 1,024 Monte Carlo paths per state. The three MCV columns form the crossed training–inference diagnostic: "w/o infer." uses an MCV-trained Actor with plain FK inference, "w/o train." uses a plain-label Actor with FK–MCV inference, and "w/o all" removes MCV from both stages. For every learned-Actor column, we report the mean and sample standard deviation of the primary RMSE over training seeds $0, 1, 2$. The fixed-A_0 configuration has no fitted Actor and hence no training-seed variability; its uncertainty instead summarizes two independent inference replicates. These absolute values make the scale of each normalized degradation explicit.

Tangent-update trajectory. Table 5 reports the absolute RMSE after each of the four learned tangent updates. Here A_0 is the fixed terminal-based initialization and A_j is the Actor after update j. Within each seed, all five Actors are evaluated on the same frozen test bank and with common Brownian random numbers. The A_0 Actor is identical across seeds, so its reported spread reflects inference-stream variability only; for $A_1, \ldots, A_4$, it also includes training-seed variability. This paired iteration diagnostic uses its own evaluation streams, so its A_0 value can differ slightly from the independently replicated "w/o tangent updates" entry in Table 4.

D Benchmark PDEs and Reference Solutions

We describe the eight PDEs in Table 1, including their terminal conditions and reference solutions. Unless otherwise specified, the spatial operator is Δ, the diffusion coefficient is $\sigma = \sqrt{2}I_d$, and $z_u = \sqrt{2}\nabla_x u$. All problems are solved backward from $t = T$. The dimensions are $d \in \{20, 50, 100, 200, 400\}$.

Radial Log-Terminal HJB. We consider

$$\begin{aligned} u_t + \Delta u - \|\nabla u\|^2 = 0, \qquad T = 1, \\ u(T,x) = \log\left(\frac{1+\|x\|^2}{2}\right). \end{aligned} \tag{68}$$

The Hopf–Cole transform gives the reference

$$\begin{aligned} u(t,x) = -\log \mathbb{E}\left[\frac{2}{1+\|X\|^2}\right], \\ X = x + \sqrt{2(T-t)}\,\xi, \quad \xi \sim \mathcal{N}(0, I_d). \end{aligned}$$

Rotational symmetry reduces its numerical evaluation to one-dimensional quadrature.

OU–Gaussian Mixture HJB. This benchmark uses $\sigma = I_d$ and a linear drift:

$$\begin{aligned} u_t + \tfrac{1}{2}\Delta u + x^\top \nabla u - \tfrac{1}{2}\|\nabla u\|^2 - d = 0, \\ u(T,x) = -\log p_0(x), \qquad T = 1. \end{aligned} \tag{69}$$

The terminal density p_0 is a frozen five-component Gaussian mixture. Let $\{\pi_k, m_k\}_{k=1}^5$ be its weights and means, with $\sum_k \pi_k = 1$, and write $\mathcal{N}_d(x; m, V)$ for the d-dimensional Gaussian density. The exact reference is $u(t,x) = -\log p_{T-t}(x)$, where

$$\begin{aligned} p_\tau(x) = \sum_{k=1}^{5} \pi_k \, \mathcal{N}_d(x; e^{-\tau} m_k, v_\tau I_d), \\ v_\tau = 2e^{-2\tau} + \tfrac{1}{2}(1 - e^{-2\tau}). \end{aligned}$$

Concave Cubic–Quadratic PDE. Let $b_1, b_2 \in \mathbb{R}^d$ be fixed orthonormal vectors, and define

$$g_{\mathrm{trig}}(x) = \tfrac{1}{2} + 0.15\big(\sin(b_1^\top x) + \sin(b_2^\top x)\big).$$

We solve

$$\begin{aligned} u_t + \Delta u + u - u^3 - \tfrac{1}{2}\|z_u\|^2 = 0, \\ u(T,x) = g_{\mathrm{trig}}(x), \qquad T = 0.5. \end{aligned} \tag{70}$$

The driver is jointly concave in (u, z_u) on the relevant bounded value interval. An independent periodic two-dimensional Fourier solver provides the reference in coordinates $(b_1^\top x, b_2^\top x)$.

Convex Cubic–Quadratic PDE. With the same form of terminal condition, we reverse the signs of the nonlinear terms:

$$\begin{aligned} u_t + \Delta u + u^3 - u + \tfrac{1}{2}\|z_u\|^2 = 0, \\ u(T,x) = g_{\mathrm{trig}}(x), \qquad T = 0.5. \end{aligned} \tag{71}$$

Its driver is jointly convex on the corresponding bounded interval. We obtain the reference from an independently refined two-dimensional spectral solve; the implementation may equivalently use the change of variables $w = -u$.

Oscillatory Value–Gradient PDE. This nonconvex–nonconcave example couples the solution and its gradient:

$$\begin{aligned} u_t + \Delta u + 0.2\sin(2\pi u) - \tfrac{1}{2}\|z_u + u b_1\|^2 = 0, \\ u(T,x) = g_{\mathrm{osc}}(x), \end{aligned} \tag{72}$$

where $T = 0.5$ and

$$g_{\mathrm{osc}}(x) = \tfrac{1}{2} + 0.3\sin(b_1^\top x)\cos(b_2^\top x).$$

The fixed orthonormal pair is specific to this PDE; its reference is computed by an independently refined two-dimensional Fourier solver.

Allen–Cahn. The value-only semilinear benchmark is

$$\begin{aligned} u_t + \Delta u + u - u^3 = 0, \qquad T = 0.3, \\ u(T,x) = \frac{1}{2 + 0.4\|x\|^2}. \end{aligned} \tag{73}$$

By radial symmetry, $u(t,x) = U(t,s)$ with $s = \|x\|^2$ solves

$$U_t + 4sU_{ss} + 2dU_s + U - U^3 = 0.$$

We compute dimension-specific radial references using spatial and temporal refinement and Richardson extrapolation.

Full-Rank Log-Sum-Exp HJB. For the full-rank benchmark, let $K = 2d$ and use fixed vectors $b_1, \dots, b_K \in \mathbb{R}^d$:

$$\begin{aligned} u_t + \Delta u - \tfrac{1}{2}\|z_u\|^2 = 0, \qquad T = 0.25, \\ u(T,x) = -\log\left(\frac{1}{K}\sum_{k=1}^{K} e^{b_k^\top x}\right). \end{aligned} \tag{74}$$

These b_k are distinct from the two-direction bases above. They are generated once and frozen, using $b_k = \gamma v + \beta \lambda_k e_k$, where $\|v\| = \|e_k\| = 1$ and $\lambda_k \in [0.5, 1.5]$. The Hopf–Cole solution is available in closed form:

$$u(t,x) = -\log\left[\frac{1}{K}\sum_{k=1}^{K} \exp\big(b_k^\top x + (T-t)\|b_k\|^2\big)\right].$$

The frozen dimension-specific (β, γ) pairs are $(1,1)$, $(1.5,1)$, $(1.5,1)$, $(2,1)$, and $(2,1)$ for $d = 20, 50, 100, 200, 400$, respectively.

Fourier–Tanh Coupled PDE. We consider

$$\begin{aligned} u_t + \Delta u + \tfrac{1}{2}\tanh(u)\|z_u\|^2 = 0, \\ u(T,x) = \operatorname{arsinh}(F_T(x)), \quad T = 0.5, \end{aligned} \tag{75}$$

with $2d$ frozen dense Fourier directions ω_k of unit norm, signs $a_k \in \{-1, 1\}$, and phases ϕ_k. Define

$$F_t(x) = \tfrac{1}{2} + \frac{2}{\sqrt{2d}} \sum_{k=1}^{2d} a_k e^{-(T-t)} \sin(\omega_k^\top x + \phi_k).$$

The substitution $F = \sinh(u)$ reduces the PDE to the heat equation. Consequently, $u(t,x) = \operatorname{arsinh}(F_t(x))$ is its analytic reference.

Evaluation and reference usage. Test states are drawn from frozen, benchmark-specific banks. The radial Log-Terminal HJB and the two-direction coupled PDEs use Gaussian spatial states; the OU–GMM states follow its source-specific time-dependent Gaussian law; Allen–Cahn uses Brownian-marginal states; and the Full-Rank and Fourier–Tanh benchmarks use uniform states in $[-1, 1]^d$. Analytic expressions and independently refined numerical references are used for evaluation and, where predeclared, benchmark calibration, but never as nonterminal actor-supervision labels or for checkpoint selection. All learned methods receive the full ambient state x; known low-dimensional structure is used only by independent reference computations.

Table 3: Baseline accuracy under increased computational budgets on two 100D PDEs. Each cell reports $10^3 \times$RMSE (mean $\pm$ seed SD); the second line gives varied hyperparameter / training time (min). C0: iteration budget; C1: stochastic-estimation budget; C2: optimization, sampling, or discretization budget.

		Full-Rank Log-Sum-Exp HJB			Fourier–Tanh Coupled PDE		
Axis	**Method**	**L1**	**L2**	**L3**	**L1**	**L2**	**L3**
C0	DPI	137.4 ± 12.9 80 / 6.2	148.9 ± 65.8 160 / 13.1	184.3 ± 67.9 320 / 24.4	110.8 ± 33.3 80 / 4.7	99.4 ± 61.1 160 / 9.3	63.2 ± 14.6 320 / 18.5
	PINN–HTE	217.9 ± 54.3 30k / 3.3	120.0 ± 21.8 60k / 6.6	186.2 ± 52.4 120k / 13.2	70.3 ± 9.5 30k / 3.4	77.3 ± 13.4 60k / 6.7	87.7 ± 8.6 120k / 13.5
	Deep BSDE	164.8 ± 0.9 1k / 21.5	160.1 ± 1.0 2k / 43.0	144.7 ± 0.6 4k / 85.8	76.7 ± 3.3 1k / 22.6	74.1 ± 0.8 2k / 45.1	59.5 ± 1.4 4k / 89.9
C1	DPI	161.6 ± 39.6 512 / 5.1	143.9 ± 16.6 16,384 / 7.6	132.8 ± 25.9 32,768 / 10.9	76.7 ± 3.8 512 / 4.1	93.7 ± 30.2 16,384 / 8.2	115.2 ± 61.1 32,768 / 8.6
	PINN–HTE	155.9 ± 88.2 64 / 7.1	205.9 ± 147.9 128 / 8.6	199.6 ± 18.1 256 / 26.6	103.5 ± 38.0 4 / 3.4	87.5 ± 11.3 64 / 7.3	76.9 ± 5.4 128 / 8.6
	Deep BSDE	161.2 ± 1.5 32 / 42.2	160.1 ± 1.0 64 / 43.0	158.1 ± 0.3 128 / 45.9	74.6 ± 1.9 32 / 44.5	74.1 ± 0.8 64 / 45.1	66.9 ± 0.8 128 / 44.3
C2	DPI	142.7 ± 18.0 4 / 3.3	155.3 ± 4.9 32 / 6.2	108.2 ± 20.9 64 / 9.6	110.8 ± 33.3 16 / 5.0	97.4 ± 23.7 64 / 9.4	63.9 ± 8.8 128 / 16.2
	PINN–HTE	217.9 ± 54.3 512 / 3.4	123.3 ± 46.7 2,048 / 12.8	136.1 ± 28.4 4,096 / 25.6	93.7 ± 20.1 256 / 3.3	77.9 ± 19.7 2,048 / 13.4	68.1 ± 10.3 4,096 / 25.8
	Deep BSDE	147.1 ± 0.7 10 / 22.1	160.1 ± 1.0 20 / 43.0	167.1 ± 0.8 40 / 81.9	65.0 ± 1.6 10 / 23.2	74.1 ± 0.8 20 / 45.1	76.7 ± 2.9 40 / 88.2

Table 4: Absolute whole-function value RMSE underlying the component-ablation figure at $d = 100$. Values are mean $\pm$ sample standard deviation. The † entries summarize two independent inference replicates rather than training seeds.

		MCV ablations				
PDE	Full TA	w/o infer.	w/o train.	w/o all	w/o terminal anchoring	w/o tangent updates†
Radial Log-Terminal HJB	$\mathbf{8.68e{-4} \pm 5.63e{-5}}$	$3.03e{-3} \pm 4.75e{-5}$	$8.85e{-4} \pm 5.20e{-5}$	$3.04e{-3} \pm 5.58e{-5}$	$1.01e{-3} \pm 9.35e{-5}$	$2.75e{-3} \pm 3.35e{-5}$
Concave Cubic–Quadratic	$\mathbf{3.42e{-4} \pm 9.78e{-6}}$	$1.85e{-3} \pm 1.48e{-5}$	$3.56e{-4} \pm 9.59e{-6}$	$1.85e{-3} \pm 1.51e{-5}$	$3.56e{-4} \pm 1.10e{-5}$	$3.35e{-3} \pm 2.38e{-5}$
Convex Cubic–Quadratic	$\mathbf{2.23e{-4} \pm 5.02e{-6}}$	$1.56e{-3} \pm 7.53e{-5}$	$2.30e{-4} \pm 5.24e{-6}$	$1.56e{-3} \pm 7.41e{-5}$	$2.34e{-4} \pm 7.93e{-6}$	$5.14e{-3} \pm 1.64e{-5}$
Oscillatory Value–Gradient	$\mathbf{8.58e{-4} \pm 4.67e{-5}}$	$1.91e{-3} \pm 5.21e{-5}$	$8.96e{-4} \pm 3.70e{-5}$	$1.93e{-3} \pm 4.86e{-5}$	$9.01e{-4} \pm 7.22e{-5}$	$6.44e{-3} \pm 1.56e{-6}$

Table 5: Absolute whole-function value RMSE along the Tangent-Actor iteration trajectory at $d = 100$. Values are mean $\pm$ sample standard deviation over the three paired seed-indexed evaluations.

	Initial tangent	Learned tangent updates			
PDE	A_0	A_1	A_2	A_3	A_4
Radial Log-Terminal HJB	$2.717e{-3} \pm 1.543e{-5}$	$1.539e{-3} \pm 1.311e{-4}$	$1.093e{-3} \pm 4.434e{-5}$	$9.822e{-4} \pm 9.366e{-5}$	$\mathbf{8.677e{-4} \pm 5.630e{-5}}$
Concave Cubic–Quadratic	$3.340e{-3} \pm 2.268e{-5}$	$3.633e{-4} \pm 1.192e{-5}$	$3.463e{-4} \pm 1.037e{-5}$	$3.423e{-4} \pm 8.804e{-6}$	$\mathbf{3.417e{-4} \pm 9.862e{-6}}$
Convex Cubic–Quadratic	$5.147e{-3} \pm 1.521e{-5}$	$2.387e{-4} \pm 4.012e{-6}$	$2.262e{-4} \pm 3.208e{-6}$	$2.240e{-4} \pm 3.513e{-6}$	$\mathbf{2.232e{-4} \pm 5.019e{-6}}$
Oscillatory Value–Gradient	$6.424e{-3} \pm 3.150e{-5}$	$1.642e{-3} \pm 9.988e{-5}$	$9.878e{-4} \pm 1.614e{-4}$	$8.848e{-4} \pm 4.874e{-5}$	$\mathbf{8.579e{-4} \pm 4.671e{-5}}$

Table 6: Final component ablation at $d = 100$. Entries are whole-function value RMSE (mean $\pm$ sample standard deviation across three training seeds). For the deterministic fixed-A_0 row, the two values are independent inference replicates rather than training seeds.

Method	Radial Log-Terminal HJB	Concave Cubic–Quadratic	Convex Cubic–Quadratic	Oscillatory Value–Gradient
Tangent-Actor (Full)	$\mathbf{8.68e-04 \pm 5.63e-05}$	$\mathbf{3.42e-04 \pm 9.78e-06}$	$\mathbf{2.23e-04 \pm 5.02e-06}$	$\mathbf{8.58e-04 \pm 4.67e-05}$
w/o MCV	$3.04e-03 \pm 5.58e-05$	$1.85e-03 \pm 1.51e-05$	$1.56e-03 \pm 7.41e-05$	$1.93e-03 \pm 4.86e-05$
w/o Terminal Anchoring	$1.01e-03 \pm 9.35e-05$	$3.56e-04 \pm 1.10e-05$	$2.34e-04 \pm 7.93e-06$	$9.01e-04 \pm 7.22e-05$
w/o Tangent Updates	$2.75e-03 \pm 3.35e-05$	$3.35e-03 \pm 2.38e-05$	$5.14e-03 \pm 1.64e-05$	$6.44e-03 \pm 1.56e-06$

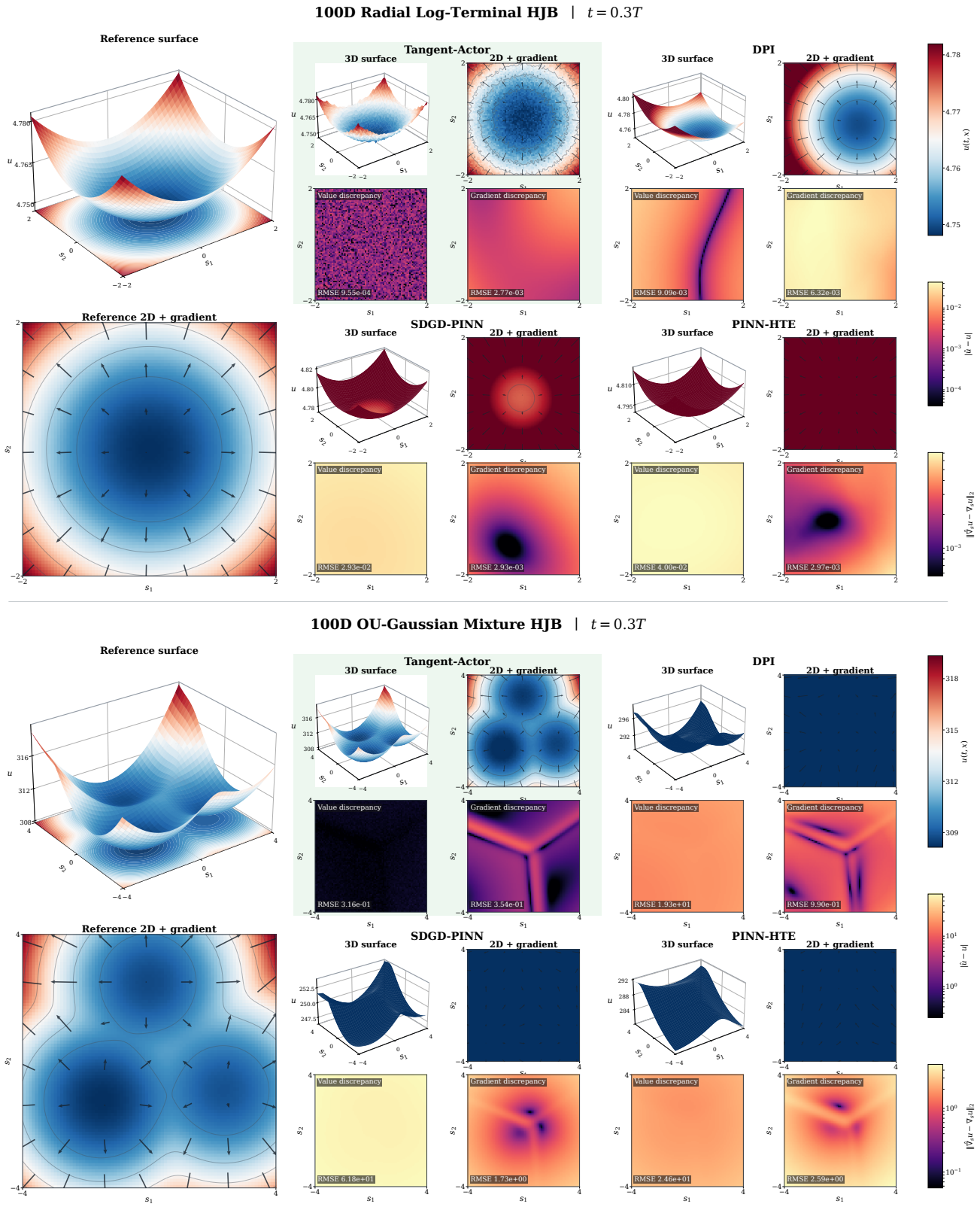


Figure 9: Solution landscapes and errors on four high-dimensional PDEs.

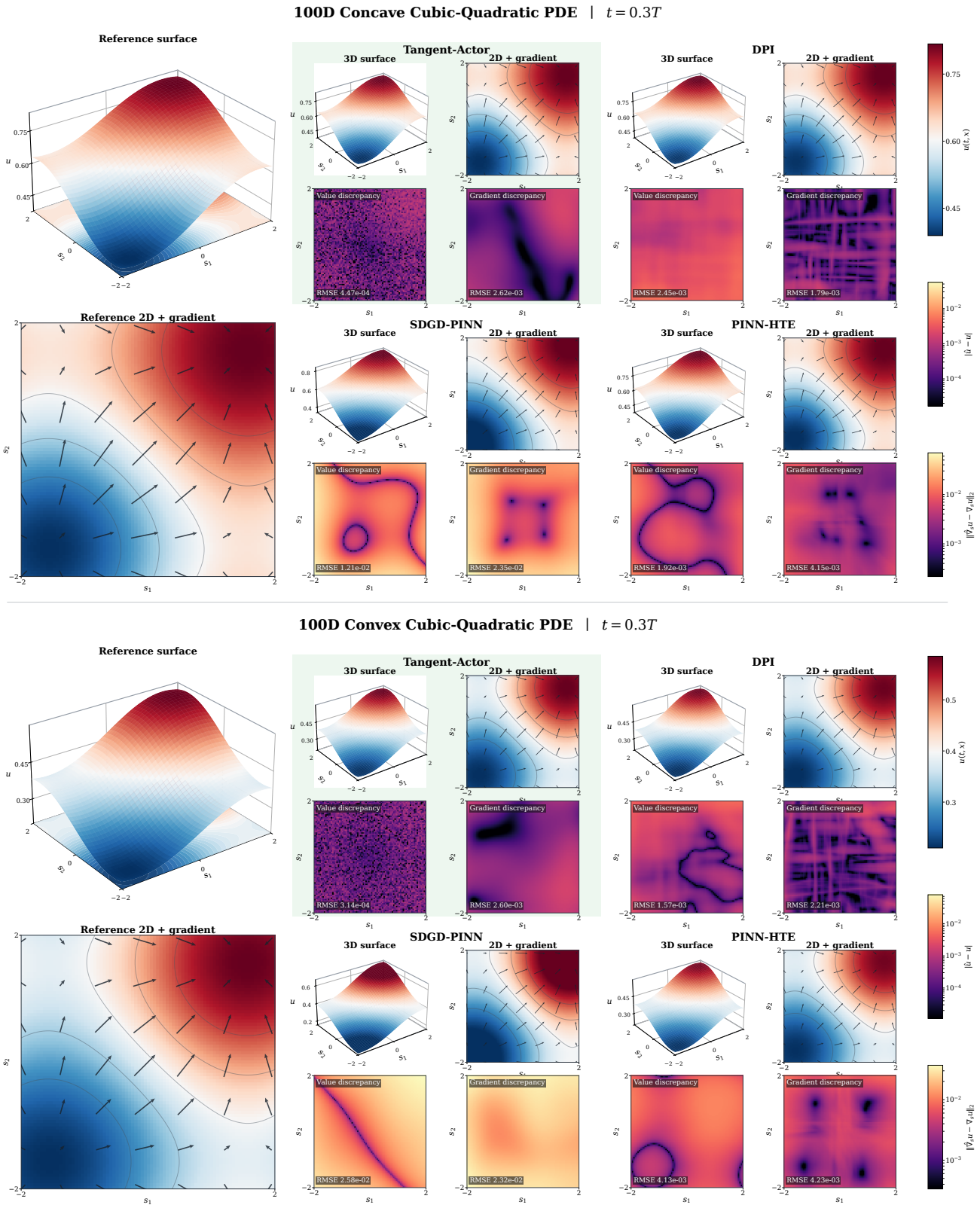


Figure 10: Solution landscapes and errors on four high-dimensional PDEs.

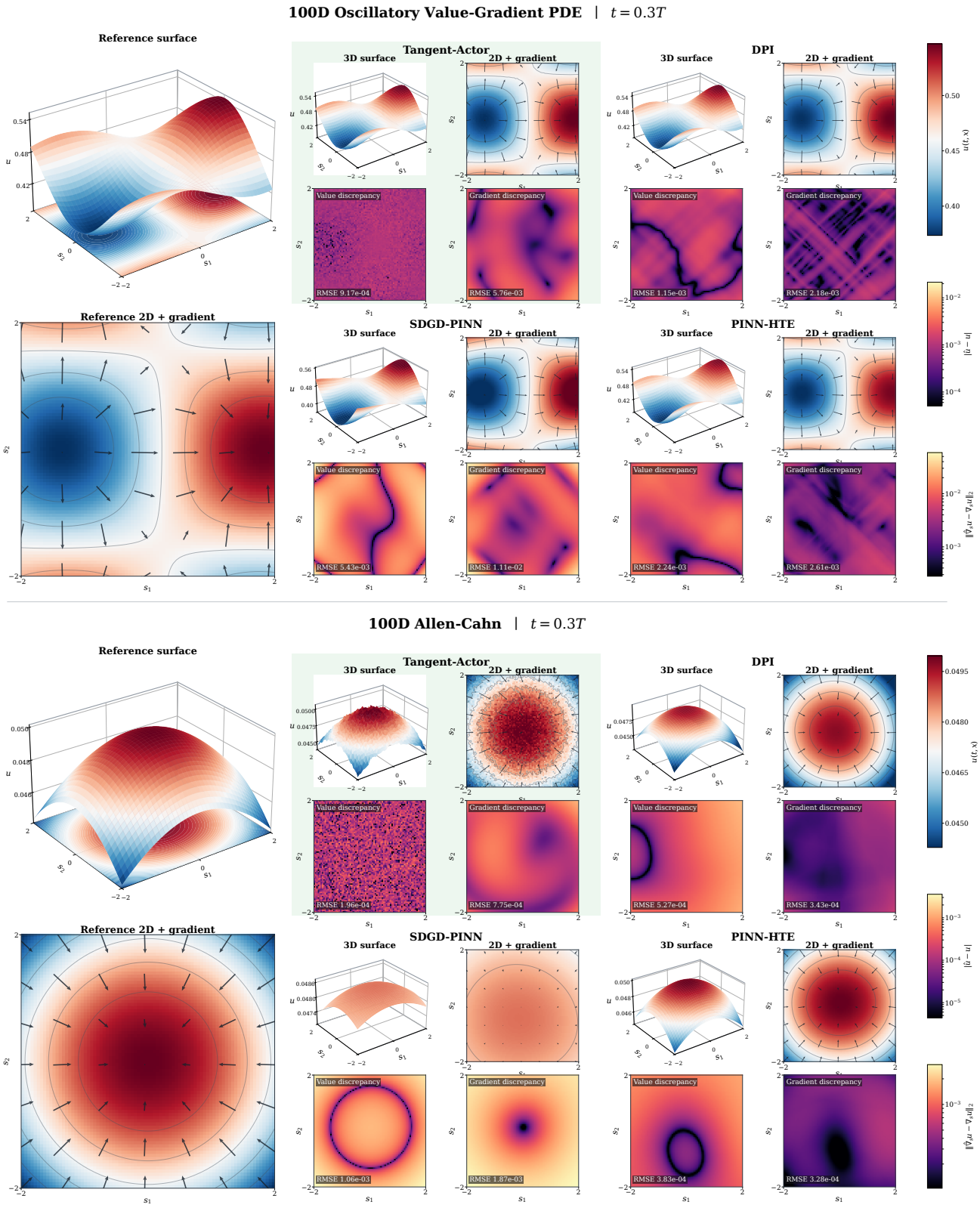


Figure 11: Solution landscapes and errors on four high-dimensional PDEs.

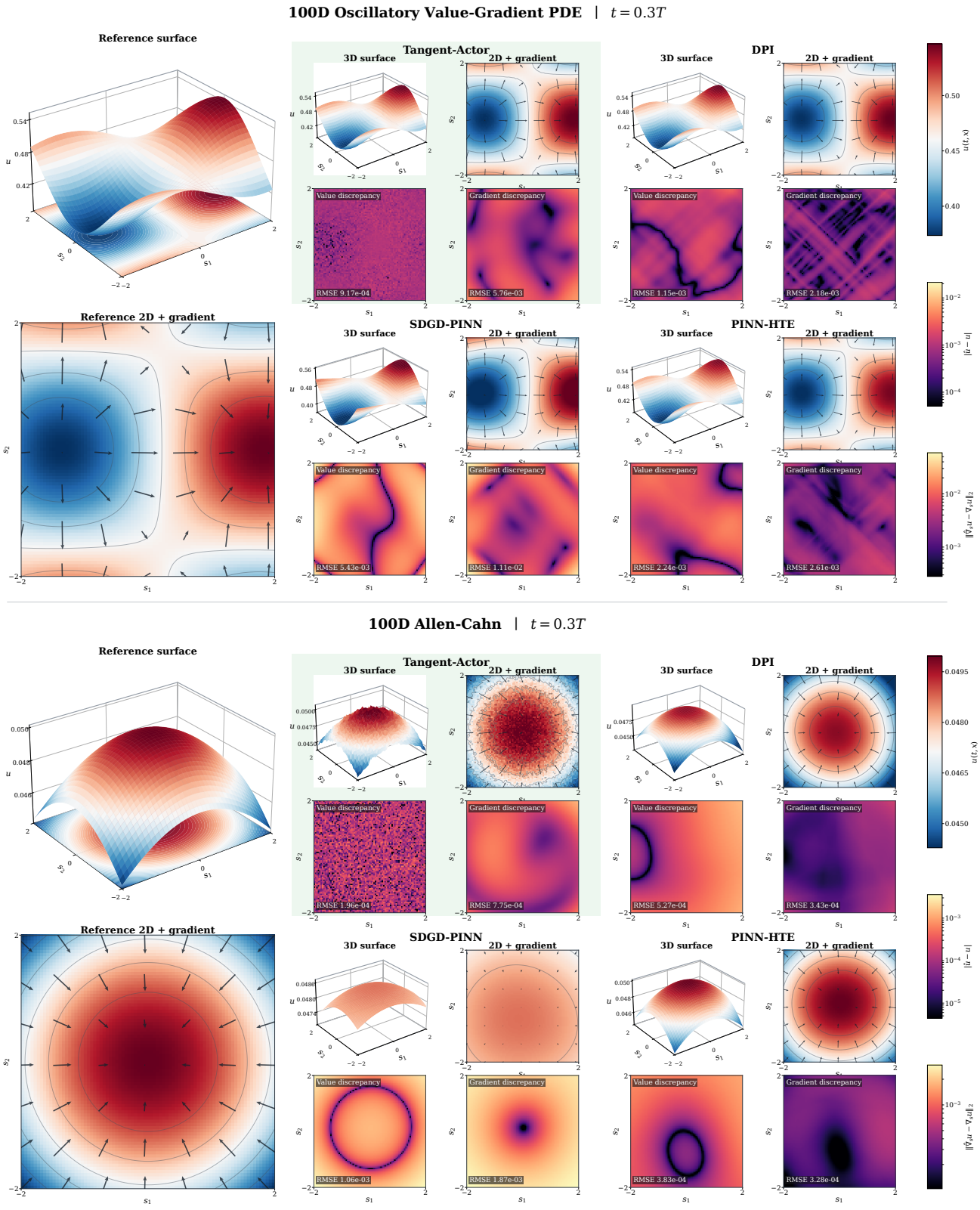


Figure 12: Solution landscapes and errors on four high-dimensional PDEs.